THE OFFICIAL FIELD MANUAL · VOLUME 1 · 2026

LD DEBLOAT GUIDE

How to strip **LDPlayer 9** down to a clean, fast, silent Android emulator — and keep it that way **forever**, on every update, on every launch, on every machine like yours.



EVERY STEP.
EVERY TRICK.
EVERY ERROR SOLVED.

14	16	20	2	875	6
SECTIONS	STEPS, 0–13	ERRORS SOLVED	VERIFIERS	MB VAULTED	LAYERS CLEAN

SCENARIO	Windows 10 (incl. Lite build) · LDPlayer 9.5.41.1 · Android 9 base · one instance
BUILT FROM	A real, complete debloat performed on this exact machine — every command printed here was executed and measured.
FINAL STATE	25/24 Windows checks PASS · 11/11 Android checks PASS · 0 LD packages enabled · intercept hijack dead · self-healing at every logon & every launch
REQUIRES	Zero prior knowledge — or administrator rights. (OK, one of those.)

Contents

EVERY SECTION, IN ORDER · 14 PARTS · READ §1 FIRST

01 **Read this first**
What you are about to do & why it works

02 **What LDPlayer hides on your PC**
The evidence board · 16 findings

03 **The Golden Rules**
The 8 rules that keep you out of trouble

04 **Meet your machine**
Check yours in 2 minutes · the RAM rule

05 **The Steps, 0–13**
The complete debloat · every command verified

06 **Errors & fixes**
19 entries from the field · the two error classes

07 **Your new toolkit**
Every script this creates & when to run it

08 **Difficulty × time × risk**
The full matrix at a glance

09 **Glossary**
Plain English · for total beginners

010 **FAQ**
15 honest questions & answers

011 **Windows 10 (debloated/Lite build)**
What was touched (nothing) & what was added

012 **The case study**
Everything that happened to the test rig, in order

013 **The printable checklist**
Tape it to the monitor

014 **Appendix — receipts**
Every change, every file, every number, verbatim

START HERE

Read this first 01

What you are about to do

You are going to remove, disable or wall off every piece of LDPlayer 9 whose job is **advertising at you, reporting on you, or selling you promoted games** — and leave everything whose job is **running your games well**. Nothing else. No guessing. Every removal is **a move, never a delete**, and the whole result **re-applies itself** at every logon and every launch.

What you need

- A Windows PC with LDPlayer 9 installed at `C:\LDPlayer\LDPlayer9` (paths in this guide assume that layout).
- Administrator rights — *or* the ability to right-click and choose "Run as administrator" (the scripts self-elevate and tell you).
- About 90 minutes the first time. The actual hands-on work is ~15 minutes; the rest is understanding, so it never needs doing again.

What you will end with

✓ **A clean machine.** No LDPlayer ads, no pop-ups, no crash reporter screaming to Huawei's servers, no app-store install redirects, no auto-start service — and a tuned 3-CPU / 3072 MB instance that survives every LDPlayer update by healing itself.

! **Honesty first.** LDPlayer is free software paid for by ads and promoted games. This guide does not hack anything — it removes *extra files we can point at* and disables *LDPlayer's own switches*. If LDPlayer truly needed those files to run games, the emulator would not boot without them. It boots. They are not needed.

THE ENEMY, IDENTIFIED

What LDPlayer hides on your PC 02

Everything below was **physically found on the test machine on release day**. Not theory — the evidence board. Read it once; then you will never quite trust a launcher again.

#	WHAT IT IS	WHERE IT LIVED	WHAT IT DID
1	**ldplayerpartner.exe** & **ldplayerpartner2.exe** (two copies, 5.3 MB each)	`C:\LDPlayer\LDPlayer9\`	The desktop pop-up ad launcher — ran during testing, logged as "player partner end"
2	**bugreport.exe** (two copies: 0.42 MB + 0.42 MB)	`LDPlayer9\` *and* `ldmutiplayer\`	Crash reporter that also ships telemetry; the manager's copy was found only by a full-tree audit
3	**dnrepairer.exe** (43.9 MB)	`LDPlayer9\`	"Repair" tool that re-downloads all the bloat it pretends to fix
4	**LDPlayer_0900054101.exe** (802.5 MB)	`LDPlayer9\`	Installer stub left behind; re-downloaded by the 9.5.41.1 update
5	**en.mp4** (4.1 MB) + **23 launcherskin images**	`LDPlayer9\data\`, `launcherskin\`	Promo video / ad skins shown in the launcher
6	**upload** — **ldupload.exe** + **libeSDKOBS.dll** + **eSDKLogAPI.dll** + **huaweisecurec.dll**	`LDPlayer9\upload\`	Huawei OBS crash-telemetry SDK — packages and ships crash data to Huawei's servers
7	**ld_launcher intercept feed** (~460 KB JSON)	`Documents\XuanZhi9\Applications\ld_launcher\`	Maps Google Play packages → **LD-store twins** (`interceptTiming: google_play_install`) — hijacks Play installs into LD-store
8	**cpi.data**	`Documents\XuanZhi9\Applications\`	CPI (cost-per-install) ad-slot map: *LauncherBottom / AppStoreSearch / AppStoreMain / AppStoreDetail*
9	**7de09267d3...** (hex-stamp file)	`Documents\XuanZhi9\Applications\`	Fetch date-stamp for the ad feed (the test rig: `release day`)
10	**recommendConfigs\dir_*.dir** + **package_configs_1005(.rcc)**	`LDPlayer9\vms\recommendConfigs\`	Key-mapping recommendation DB with `advertising: true` baked into suggestions
11	**LDPlayerENSvr** (ldplayerserviceEN.exe)	Windows service, *Automatic*	Auto-start helper; at 05:08 today it launched the multi-instance manager before you clicked anything
12	**the partner log**	`LDPlayer9\log\`	Telemetry ledger: a successful report POST was observed during testing

#	WHAT IT IS	WHERE IT LIVED	WHAT IT DID
13	**Telemetry pins**	`vms\config\leidians.config`	`reportPCInfoTime=20261110`, `adLoadtime`, `nextCheckupdateTime` — all re-armed to today by the update
14	**partner_report.data**	`%APPDATA%\XuanZhi9\`	10-byte ASCII timestamp of the last successful report (`1794335940` = today)
15	**cache** (ldrescdn.com 1.1 MB ad image, fake *com.google.android.gms_icon.png*, cpt_ad.data)	`%APPDATA%\XuanZhi9\cache\`	Live ad payload that re-downloaded after the update
16	**launch-time re-download**	*every launch of the emulator*	Intercept feed + recommend DB re-fetched **~2 minutes after every start**; pins re-armed. Built into the launcher lifecycle

! **Why "just delete the exes" was never enough.** Items 7–10 and 16 mean LDPlayer also holds **feed URLs, slot maps and fetch stamps** that refresh themselves on their own schedule. That is why the permanent fix is *layers* — pins, seal, firewall, disabled packages, logon task — not a one-off cleanup.

READ TWICE

The Golden Rules 03

★ **RULE 1 Ã¢â‚¬â€ Rename first. Never delete.** Every removal in this guide is **a move**, never a delete. There is no "go back" line item here Ã¢â‚¬â€ the only direction in this document is forward, and attempting to recreate the bloat is not a supported workflow.

★ **RULE 2 — Read the logs, never guess.** The boot freezes, the service refusal, the CPU mystery — every one was solved by **reading log files** (VBox.log, the partner log, the VBox error text). Feelings are not diagnostics.

★ **RULE 3 — Measure before you conclude.** "The debloat broke the emulator" was wrong. Measurement showed the environment was starved — the debloat was innocent. **Capture the number first.**

★ **RULE 4 — Bisect with last-known-good.** 4096 MB boots / 2048 MB freezes / 3072 MB chosen. *Data, not opinion.*

★ **RULE 5 — Verify before you destroy.** The Android-9 `--brief` flag printed garbage; the script safely refused to continue rather than break the home screen. **If a check is unclear, stop.** "It looked odd" beats "it's probably fine", every time.

★ **RULE 6 — Never strip ACLs without setting replacements in the same move.** The ad-cache seal DID go wrong once — the folder became unwritable even for us. The failure taught a law: **plan your replacement before you change anything.** What you break carelessly, you keep broken.

★ **RULE 7 — Pattern-match, never hardcode.** We found `com.ldmnq.launcher3` by *searching*, not by assuming the standard launcher name. Names lie; patterns catch.

★ **RULE 8 — Updates undo debloat. Plan for it.** The 9.5.41.1 update silently restored everything (CPU 3→4, bloat, telemetry, the 802 MB stub) — **including files someone had already carefully removed.** That is why the fix lives in a **logon task**, why the launch needs its own **watcher**, and why `verify-debloat.ps1` exists.

KNOW YOUR BATTLEFIELD

Meet your machine 04

The machine this guide was built on (match yours)

SPEC	VALUE
GPU	integrated graphics (integrated)
OS	Windows 10 Pro 2009 (build 19041.5553), a **Lite/debloated** build (`WdNisSvc` already disabled). See §11.
LDPlayer	9.5.41.1 (build 0900054101), Android 9 base, one instance **leidian0** (“LDPlayer”), 1600×900
Shared folders	`Documents\XuanZhi9\{Applications, Pictures, Misc}`

Check yours in 2 minutes (paste into PowerShell)

LD · COMMAND

```
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion
(Get-Item "C:\LDPlayer\LDPlayer9\dnplayer.exe").VersionInfo.ProductVersion
Get-CimInstance Win32_OperatingSystem | Select-Object @{n="FreeGB";e=
{[math]::Round($_.FreePhysicalMemory/1MB,2)}}
Get-PSDrive C | Select-Object @{n="FreeGB";e={[math]::Round($_.Free/1GB,1)}}
```

ℹ **Constrained machine?** Give the guest 4096 MB only if boot *and* day-to-day play stay smooth. Whenever the box is tight on memory, **3072 MB is the rule.**

THE MEAT

5 · THE STEPS, 0–13

Difficulty legend: EASY = anyone · MEDIUM = follow exactly, it will work · HARD = the steps where old builds break · EXPERT = ADB/Android internals.

Every command shown was executed verbatim on the test machine. Copy with confidence.

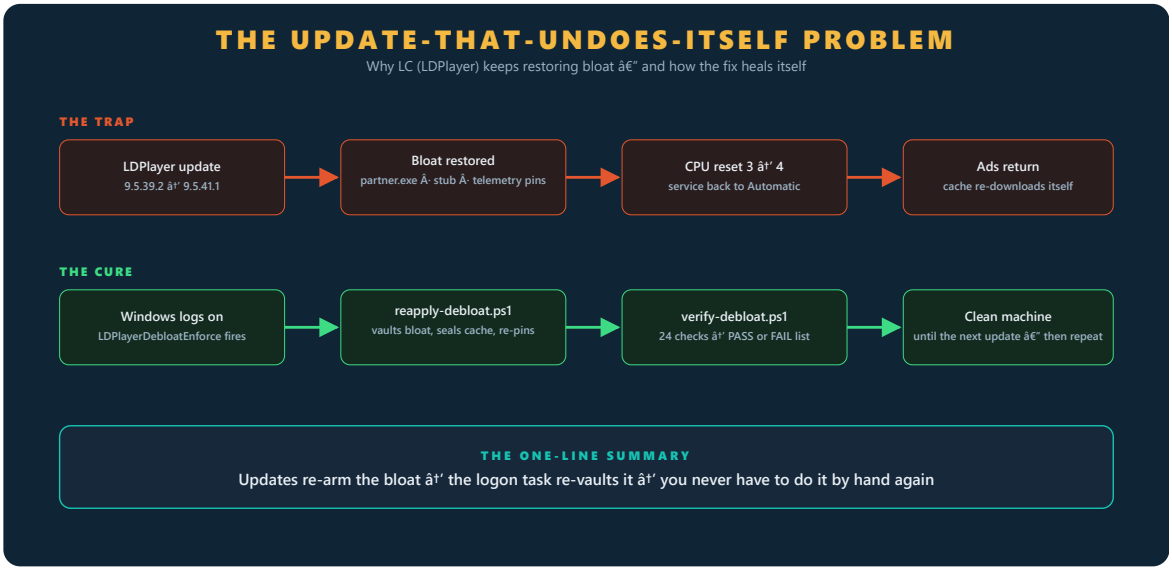


FIG. 1 - THE UPDATE-THAT-UNDOES-ITSELF PROBLEM, AND THE FIX THAT HEALS IT.

0 The 30-second symptom check

EASY | ~1 min | risk: none

WHY

Debloating the wrong problem wastes an hour. Confirm LDPlayer is your issue, not Windows.

LOOK FOR THESE SIGNS

- LDPlayer window shows ads, a featured-games wall, or app-store suggestions you never asked for.
- Pop-ups on the **Windows desktop** when LDPlayer is closed or at startup.
- Fan ramps up while idle; memory sits above 85%; LDPlayer boots slowly or "freezes" mid-boot.
- Promotional video or skin carousel inside the launcher.

IF YES

Continue to Step 1. If none apply, close this guide — you do not have the disease it treats.

1 Know your machine (and your LDPlayer)

EASY | ~2 min | risk: none

DO THIS — paste into PowerShell

LD · COMMAND

```
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion
(Get-Item "C:\LDPlayer\LDPlayer9\dnplayer.exe").VersionInfo.ProductVersion
Get-CimInstance Win32_OperatingSystem | Select-Object @{n="FreeGB";e=
{[math]::Round($_.FreePhysicalMemory/1MB,2)}}
Get-CimInstance Win32_ComputerSystem | Select-Object @{n="TotalGB";e=
{[math]::Round($_.TotalPhysicalMemory/1GB,2)}}
Get-PSDrive C | Select-Object @{n="FreeGB";e={[math]::Round($_.Free/1GB,1)}}
```

WHAT YOU MUST NOTE DOWN

- Your **LDPlayer version** (ours: 9.5.41.1) — first thing any fix support will ask for.
- **Instance state** — `ldconsole list2` prints its index, name, DPI and resolution — the ground truth, straight from the manager.
- **Disk headroom** — enough for the vault; LD ships a full installer stub per update you decline.

EXPECTED RESULT

A small note with: LDPlayer version, instance index, resolution. That is your baseline for verifying the end state.

2 Photo the evidence BEFORE changing anything

EASY | ~5 min | risk: none — protects everything after it

WHY

Two reasons. (1) If something looks wrong later, you can prove what was original. (2) It forces you to look at the **actual files** before trusting any checklist — including this one.

DO THIS — capture the five things

LD · COMMAND

```
Get-ChildItem "C:\LDPlayer\LDPlayer9" | Sort-Object Name | Select-Object Name, Length
Get-Content "C:\LDPlayer\LDPlayer9\vms\config\leidians.config"
Get-Content "C:\LDPlayer\LDPlayer9\vms\config\leidian0.config"
Get-Service LDPlayerENSvr
New-Item -ItemType Directory -Force -Path "C:\LDPlayer\vault" | Out-Null
Get-Item "C:\Windows\System32\drivers\etc\hosts" | Select-Object FullName, Length
```

SAVE THESE

- The file listing (redirect it to a .txt if you want a paper trail).
- The two config files, quoted verbatim.
- A screenshot of the launcher **as it looks today** — ad skins and recommendation walls. Proof it existed, for "it came back?!" moments later.

ON the test rig

These five captures are what proved the 9.5.41.1 update had restored the bloat, re-armed telemetry (its telemetry pins and reset CPU 3→4. Without them we would have been guessing.

3 Wave 1 — vault the classic bloat files

EASY | ~3 min | risk: none — moves only, zero deletes

WHAT WE DO

Move the known offenders into a timestamped vault folder. Never delete — move, verify, continue.

LD · COMMAND

```
$vault = "C:\LDPlayer\vault\$(Get-Date -f yyyyMMdd-HHmmss)"
New-Item -ItemType Directory -Force -Path $vault | Out-Null
@('C:\LDPlayer\LDPlayer9\ldplayerpartner.exe',
  'C:\LDPlayer\LDPlayer9\ldplayerpartner2.exe',
  'C:\LDPlayer\LDPlayer9\bugreport.exe',
  'C:\LDPlayer\LDPlayer9\dnrepairer.exe',
  'C:\LDPlayer\LDPlayer9\LDPlayer_*.exe') | ForEach-Object {
    Get-ChildItem $_ -ErrorAction SilentlyContinue |
      Move-Item -Destination { Join-Path $vault (Split-Path $_.FullName -Leaf) } -Force }
Move-Item "C:\LDPlayer\LDPlayer9\data\en.mp4" (Join-Path $vault "en.mp4") -Force -ErrorAction
SilentlyContinue
Move-Item "C:\LDPlayer\LDPlayer9\launcherskin" (Join-Path $vault "launcherskin") -Force -
ErrorAction SilentlyContinue
Get-ChildItem $vault
```

EXPECTED RESULT

Vault now holds: 2× ldplayerpartner (5.3 MB each), bugreport.exe, dnrepairer.exe, the 802 MB installer stub (if present), en.mp4 and the skin images. The install folder is hundreds of MB leaner. The emulator still boots — none of those files render one frame of Android.

! **WHY NOT DELETE?** LDPlayer wires loosely-coupled binaries together. Removing one outright strays from what the framework can regenerate — a parked file does no harm where a deleted one can't be reconciled.

4 Seal the ad cache — the right way this time

MEDIUM | ~5 min | risk: low — this is the step that bit us before; handled correctly now

THE HISTORY (why this step exists)

The first attempt emptied the cache and then **stripped its ACLs** — the folder became unwritable even for us. The correct order exists for a reason: **empty → add exactly one Deny ACE → prove with a real write attempt.**

DO THIS — empty + seal

LD · COMMAND

```
$cache = Join-Path $env:APPDATA "XuanZhi9\cache"
Move-Item (Join-Path $cache "*") "C:\LDPlayer\vault\$((Get-Date -f yyyyMMdd))-adcache" -Force -
ErrorAction SilentlyContinue
icacls $cache /deny "Everyone:(OI)(CI)W"
# PROOF - this MUST fail:
try { [IO.File]::WriteAllText((Join-Path $cache "__test.txt"),"x"); "SEAL FAILED" }
catch { "Write correctly denied - sealed." }
```

EXPECTED RESULT

- `Write correctly denied - sealed.` If it prints `SEAL FAILED`, your ACE did not land — rerun `icacls … /deny …` as administrator.
- LDPlayer has nowhere to write ad payloads. The cache stays empty, permanently, by ACL.

ERRORS YOU WILL SEE

- `Access is denied` on the **Move-Item** — you are not elevated. Right-click PowerShell → Run as administrator.
- Everything works but the launcher *still shows ads once* — expected. This seals the channel; it cannot unsee ads that already downloaded into the launcher's own disk cache. That will starve out by itself (nothing re-feeds it), or Step 10's logon re-seal catches the next attempt.

5 Pin telemetry & update timestamps (the quiet killer)

MEDIUM | ~4 min | risk: low — the file is a config, not the Android image

THE WHY

LDPlayer keeps three numbers in `leidians.config` that decide when it phones home and when it re-checks for updates. The 9.5.41.1 update re-armed all three to **today**. We pin them to the year 2100.

LD · COMMAND

```
$cfgPath = "C:\LDPlayer\LDPlayer9\vms\config\leidians.config"
$cfg = Get-Content $cfgPath -Raw | ConvertFrom-Json
$cfg.reportPCInfoTime    = "20990101"     # last time it reported; now "never again"
$cfg.adLoadtime          = 4102444800     # 2100-01-01
$cfg.nextCheckupdateTime = 4102444800     # 2100-01-01
$cfg.noiceUserRed        = $false         # the red notification dot
[IO.File]::WriteAllText($cfgPath, ($cfg | ConvertTo-Json -Depth 5), (New-Object
Text.ASCIIEncoding))
Get-Content $cfgPath
```

EXPECTED RESULT

The four values print back as `20990101` / `4102444800` / `4102444800` / `false`. Note the JSON formatting normalizes (spacing changes); **the keys and values are what matter** — LDPlayer parses JSON, not pretty-print.

ERRORS YOU WILL SEE

- `ConvertFrom-Json : Invalid JSON` — the file was hand-edited or a write was interrupted. LDPlayer regenerates its configs on the next launch; if it does not, re-apply Steps 3–5.
- You pin it to 2100, and an update *re-arms it anyway* — expected with LDPlayer: updates rewrite this file. That is exactly why Step 10 (logon task) re-pins it silently every time you sign in.

6 CPU & RAM tuning — the 3072 rule

HARD ~10 min + one test boot

risk: MEDIUM — get the numbers wrong and the emulator boot-loops

WHY THIS STEP IS THE HARDEST

Backwards reasoning here bricks your boot. The evidence, in order, from the logs:

- **4096 MB / 4 CPU** → the host starved; the guest swapping killed play. *Too big.*
- **2048 MB** → VM froze at the exact same spot twice (16 s after network setup). *Too small* — Android needs its first-boot headroom.
- **3072 MB / 3 CPU** → boots, stable. *The measured sweet spot.*

DO THIS — set CPU via the official tool (safest path)

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\ldconsole.exe" modify --index 0 --cpu 3
Select-String -Path "C:\LDPlayer\LDPlayer9\vms\config\leidian0.config" -Pattern cpuCount
```

DO THIS — RAM stays whatever the vbox says (do NOT hand-edit it live)

LD · COMMAND

```
Select-String -Path "C:\LDPlayer\LDPlayer9\vms\leidian0\leidian.vbox" -Pattern "RAMSize|CPU count"
# Target: RAMSize="3072" and CPU count regenerated as 3 on next launch from leidian0.config
```

! **THE vbox IS NOT AN EDITOR.** `leidian.vbox` is auto-generated from `leidian0.config` **on launch**. Hand-editing it while VirtualBox is up gets overwritten silently — and editing it with the emulator running can corrupt the VM definition. Never. Edit the config or use `ldconsole`. Wait for the next launch to see the visual effect.

i **THE 40% RULE (memorize):** never **reserve** more than ~40% of host RAM for the guest. On a memory-tight box that means **3072 MB** (the *cap*). On a large box it can mean 4096 MB. **Reservation is not usage:** LD idles at a few hundred MB committed and only climbs toward the cap when a game demands it — the freeze came from too little *headroom*, not from LD devouring 3–5 GB.

TEST

Launch LDPlayer (Step 13 note: as administrator). Watch it through the boot. If it freezes at the 16-second mark **twice identically**, your value is wrong for your machine — drop to the last value that booted and work upward in 512 MB steps, checking free RAM and `VBox.log` each time. (the test rig never went there; 3072 was home.)

7 Kill the service (LDPlayerENSvr)

MEDIUM | ~2 min | risk: low — it only relaunches the ad/telemetry plumbing

WHY

`LDPlayerENSvr` (binary: `ldplayerserviceEN.exe`) is set to **Automatic** and starts at logon; it relaunches the multi-instance manager and its ad plumbing before you even click anything. We set it to **Disabled**. It does not serve the emulator's games — it serves the business model.

LD · COMMAND

```
$svc = Get-Service LDPlayerENSvr
$svc.Status; $svc.StartType
if ($svc.Status -ne "Stopped") { Stop-Service LDPlayerENSvr -Force }
Set-Service LDPlayerENSvr -StartupType Disabled
(Get-Service LDPlayerENSvr) | Select-Object Status, StartType
```

EXPECTED RESULT

`Stopped / Disabled`. The multi-instance manager still opens when you launch LDPlayer manually (that is not the service's job); it just no longer ambushes you at every sign-in.

IF IT REFUSES

PowerShell is not elevated. Right-click → Run as administrator. There is no reason to keep this service enabled.

8 The network layers — hosts list + firewall blocks

MEDIUM | ~4 min | risk: low — additive, marked, fully reversible

TWO LAYERS, ON PURPOSE

Hosts covers the domains it talks to *today*; the **firewall** covers the executables, which survives domain rotations. Different failure modes — both cheap.

LAYER A — marked hosts block

LD · COMMAND

```
$block = @'

# LDPlayer-Debloat-Block-Begin (LDPlayer ad/telemetry endpoints)
0.0.0.0 ldrescdn.com
0.0.0.0 res.ldrescdn.com
0.0.0.0 img.ldrescdn.com
0.0.0.0 tj.ldmnq.com
0.0.0.0 ad.ldmnq.com
0.0.0.0 ads.ldplayer.net
0.0.0.0 report.ldmnq.com
0.0.0.0 alog.ldmnq.com
0.0.0.0 data.ldplayer.net
0.0.0.0 res.ldplayer.net
# LDPlayer-Debloat-Block-End
'@
[IO.File]::AppendAllText("C:\Windows\System32\drivers\etc\hosts", $block, (New-Object
Text.ASCIIEncoding))
```

LAYER B — program-scoped firewall blocks (7 rules)

LD · COMMAND

```
$rules = @{
  'LDPlayerBlock-ldplayerpartner'    = 'C:\LDPlayer\LDPlayer9\ldplayerpartner.exe'
  'LDPlayerBlock-ldplayerpartner2'   = 'C:\LDPlayer\LDPlayer9\ldplayerpartner2.exe'
  'LDPlayerBlock-dnrepairer'         = 'C:\LDPlayer\LDPlayer9\dnrepairer.exe'
  'LDPlayerBlock-bugreport'          = 'C:\LDPlayer\LDPlayer9\bugreport.exe'
  'LDPlayerBlock-ldplayerserviceEN' = 'C:\LDPlayer\LDPlayer9\ldplayerserviceEN.exe'
  'LDPlayerBlock-ldupload'           = 'C:\LDPlayer\LDPlayer9\upload\ldupload.exe'
  'LDPlayerBlock-bugreport-manager' = 'C:\LDPlayer\ldmutiplayer\bugreport.exe'
}
foreach ($n in $rules.Keys) {
  New-NetFirewallRule -DisplayName $n -Direction Outbound -Action Block -Program $rules[$n] -
Profile Any -Enabled True | Out-Null }
```

EXPECTED RESULT

`Get-NetFirewallRule -DisplayName 'LDPlayerBlock-*'` lists 7 Enabled Outbound rules. The blocked executables, even if an update restores them verbatim, cannot open a socket.

> *i* **WHY PROGRAM-SCOPED RULES BEAT DOMAIN LISTS:** hosts says "never resolve this name" — games still load, ads starve. Firewall says "this exact .exe can never phone home" — even if LD's ad CDN gets a new domain tomorrow. One layer is static; the other is surgical.

9 Wave 2 — the deep sweep that catches what Wave 1 misses

MEDIUM→HARD | ~15 min | risk: low — same rules: vault, never delete, verify after

WHY THIS EXISTS

A full tree inventory (every file, every path, changed and unchanged) found an entire second ad/telemetry stack that the "classic" playbook never looked at. If you only vault the famous five, **you have debloated the window frames and left the engine running.**

CATCH 1 — the Huawei OBS crash-telemetry SDK

LD · COMMAND

```
# libeSDKOBS.dll / eSDKLogAPI.dll / huaweisecurec.dll / ldupload.exe
Move-Item "C:\LDPlayer\LDPlayer9\upload" "C:\LDPlayer\vault\$((Get-Date -f yyyyMMdd))-upload-sdk"
-Force
```

What it did: packaged crash dumps and telemetry and shipped them to Huawei OBS storage. Not required. Blocked at the firewall now.

CATCH 2 — the app-store intercept engine (LD-store hijack)

LD · COMMAND

```
$ad = Join-Path $env:USERPROFILE "Documents\XuanZhi9\Applications"
@('ld_launcher','cpi.data','appInfo.txt') | ForEach-Object {
  Move-Item (Join-Path $ad $_) "C:\LDPlayer\vault\$((Get-Date -f yyyyMMdd))-intercept\$_" -Force -
ErrorAction SilentlyContinue }
Get-ChildItem $ad -Force | Where-Object { $_.Name -match '^[0-9a-f]{32}$' } |
  Move-Item -Destination "C:\LDPlayer\vault\$((Get-Date -f yyyyMMdd))-intercept" -Force
```

- `ld_launcher\2e102ace…` — ~460 KB JSON listing Google Play packages and their **LD-store twins**, with `interceptTiming: google_play_install`. When you install a matching game from Play, LD substitutes its own download to capture the install credit and serve you its store.
- `cpi.data` — the ad-slot map: LauncherBottom / AppStoreSearch / AppStoreMain / AppStoreDetail.
- `7de09267d3…` — fetch date-stamp.

CATCH 3 — the manager's own bugreport.exe

LD · COMMAND

```
Move-Item "C:\LDPlayer\ldmutiplayer\bugreport.exe" "C:\LDPlayer\vault\$((Get-Date -f yyyyMMdd))-
manager-bugreport.exe" -Force
```

Wave 1 vaulted the LDPlayer9 copy; this one lives in `ldmutiplayer` — found only by inventoring the whole tree.

CATCH 4 — the telemetry ledger

LD · COMMAND

```
Move-Item "C:\LDPlayer\LDPlayer9\log\the partner log" "C:\LDPlayer\vault\$((Get-Date -f
yyyyMMdd))-the partner log" -Force
```

This log proved today's telemetry POST succeeded during testing. Once vaulted, nothing appends to it.

CATCH 5 — the recommendation/ad database

LD · COMMAND

```
Get-ChildItem "C:\LDPlayer\LDPlayer9\vms\recommendConfigs" -Force |
  Where-Object { $_.Name -in
'dir_kmps.dir','dir_jmps.dir','dir_noices.dir','dir_commons.dir','package_configs_1005','package_c
onfigs_1005.rcc' } |
  Move-Item -Destination "C:\LDPlayer\vault\$((Get-Date -f yyyyMMdd))-recommendDB" -Force
```

Kept: `dynamic-key.xml` — PUBG-style smart-keymap pixel data (a feature). Move it too if you want; the emulator does not validate it.

AND THEN THE UPDATE BIT AGAIN — the at launch launch incident

On **release day at at launch–at launch**, merely **launching the emulator** re-downloaded the intercept feed and recommendation DB **within ~2 minutes of start** — and re-armed `leidians.config` (telemetry pins, update check, red-dot). The Windows-side task cannot stop that; it is wired into LDPlayer's launch lifecycle. The files land back as **inert dead weight** because the packages that consume them (app-store, LD launcher) are disabled-user. Step 13 shows the one habit that handles this forever.

EXPECTED RESULT

`verify-debloat.ps1` shows zero files for the upload SDK, manager bugreport, intercept feed, recommendation DB and partner log. All five also get re-vaulted automatically by `reapply-debloat.ps1`.

10 Permanence — the logon enforcement task

HARD | ~10 min | risk: LOW if you copy exactly; the failure mode is "it just doesn't run"

WHY THIS IS NOT OPTIONAL

LDPlayer updates re-restore everything — partner exes, stub, telemetry pins, service mode — silently, without asking. And a mere **launch** re-downloads the intercept feed. Any cleanup you do by hand **will be undone**. So the fix must live **inside Windows itself**, and re-apply at every sign-in.

THE THREE PIECES

1. **C:\LDPlayer\reapply-debloat.ps1** — the master re-fixer: vaults restored bloat, re-seals the cache, re-pins 2100 timestamps, disables the service, re-ensures hosts + 7 firewall rules, sets CPU back to 3, **and if the emulator is attached, re-applies the inside-Android state too**. Self-elevating, idempotent, logged to `super-upgrade-log.txt`.
2. **Scheduled task** `LDPlayerDebloatEnforce` — trigger: at logon; highest privileges; hidden.
3. **The Clean Launch shortcut** (Step 13) — for the launch-time re-download that no logon task can reach.

CREATE THE TASK

LD · COMMAND

```
$action = New-ScheduledTaskAction -Execute 'powershell.exe' -Argument '-NoProfile -ExecutionPolicy
Bypass -WindowStyle Hidden -File C:\LDPlayer\reapply-debloat.ps1 -Silent'
$trigger = New-ScheduledTaskTrigger -AtLogOn -User "$env:USERDOMAIN\$env:USERNAME"
$principal = New-ScheduledTaskPrincipal -UserId "$env:USERDOMAIN\$env:USERNAME" -RunLevel Highest
-LogonType Interactive
$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries -
ExecutionTimeLimit (New-TimeSpan -Minutes 5) -Hidden
Register-ScheduledTask -TaskName 'LDPlayerDebloatEnforce' -Action $action -Trigger $trigger -
Principal $principal -Settings $settings -Force
```

PROVE IT WORKS

LD · COMMAND

```
Start-ScheduledTask -TaskName 'LDPlayerDebloatEnforce'
(Get-ScheduledTaskInfo -TaskName 'LDPlayerDebloatEnforce').LastTaskResult   # must print 0
Get-Content C:\LDPlayer\super-upgrade-log.txt -Tail 5
```

> **ORDER MATTERS:** logon fires before you double-click LDPlayer. The task repairs; the emulator then starts clean. For the launch-time re-download, Step 13's Clean Launch shortcut is the other half of the answer.

11 Prove it — the verifiers (Windows + Android)

EASY | ~1 min each | risk: none — read-only

WINDOWS SIDE

LD · COMMAND

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\LDPlayer\verify-debloat.ps1
```

• • • LD CYCLE LOG

```
LDPlayer version: 9.5.41.1 | checked: release day at re-sweep:50
[PASS] bloat absent (partner×2, bugreport×2, dnrepairer, stub, promo, skins)
[PASS] upload SDK gone  [PASS] manager bugreport gone  [PASS] partner log vaulted
[PASS] app-store ad feed vaulted  [PASS] recommendation/ad DB vaulted
[PASS] ad cache sealed (write denied) [PASS] smart-keymap data kept (by design)
[PASS] telemetry pinned ×3  [PASS] red-dot nag off  [PASS] CPU = 3  [PASS] RAM = 3072 MB
[PASS] LDPlayerENSvr disabled  [PASS] hosts block  [PASS] 7 firewall blocks  [PASS] free RAM healthy
RESULT: ALL CHECKS PASSED
```

ANDROID SIDE (run while the emulator is up)

LD · COMMAND

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\LDPlayer\verify-android.ps1
```

• • • LD CYCLE LOG

```
ADB: connected | audited release day 12:1x:xx
[PASS] Lawnchair installed  [PASS] LD launcher DISABLED
[PASS] Aurora Store installed  [PASS] LD App Store DISABLED
[PASS] Magisk installed  [PASS] photo dreams DISABLED
[PASS] print spooler DISABLED  [PASS] Private DNS = dns.adguard.com
[PASS] Lawnchair is HOME  [PASS] ldmarket:// hijack DEAD
RESULT: ALL ANDROID CHECKS PASSED
```

`exit code = number of FAILs` for both.

12 Inside Android — the half nobody can see from Windows

EXPERT ~20 min

risk: MEDIUM — editing a live Android 9; but this one was done and VERIFIED end-to-end on the test rig

WHAT ACTUALLY HAPPENED HERE (evidence, not theory)

An update did not just restore files — it **re-imaged the Android side**. After the update, the home screen showed the LD launcher again, **LDStore** and **LDCloud-Cloud Phone for Roblox** as icons, and **zero third-party apps** — Lawnchair, Aurora and Magisk had all been wiped. Every in-Android customization was gone. The emulator must be treated as **reset by every update**, and the inside-Android phase must be re-observed after every re-image.

THE FIX, VERIFIED END-TO-END (release day, ~in a follow-up pass)

1 — Open the ADB door (the update locks it)

LDPlayer → **gear icon** → **Other** tab → **ADB debugging** → **Local connection: ON**. Without this, `adb devices` prints nothing and no in-Android fix can be applied — the "ADB always refused" mystery, solved by exactly this toggle.

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\adb.exe" devices
# emulator-5554   device
```

2 — Audit, don't assume (81 packages, all read)

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\adb.exe" shell pm list packages
& "C:\LDPlayer\LDPlayer9\adb.exe" shell pm list packages -3        # user apps: EMPTY (wiped)
& "C:\LDPlayer\LDPlayer9\adb.exe" shell pm list packages -d        # disabled: nothing yet
```

Findings: exactly two LD-owned packages (`com.ldmnq.launcher3`, `com.android.ld.appstore`); every third-party app deleted by the update; your **Xiaomi MI 9**-fingerprinted Android 9 image was rebuilt.

3 — Install genuine APKs (verified, not trusted) — put the APKs anywhere you like; these examples use `C:\ld-apps\`

LD · COMMAND

```
mkdir C:\ld-apps   :: skip if the folder exists
:: download the three APKs from the URLs below with your browser or Invoke-WebRequest, then:
& "C:\LDPlayer\LDPlayer9\adb.exe" install -r "C:\ld-apps\Lawnchair.apk"
& "C:\LDPlayer\LDPlayer9\adb.exe" install -r "C:\ld-apps\Aurora.apk"
& "C:\LDPlayer\LDPlayer9\adb.exe" install -r "C:\ld-apps\Magisk.apk"
```

All three verified with LDPlayer's own `aapt` first (package names match the genuine apps). Download URLs:

- Lawnchair 1.2.0 — `https://github.com/LawnchairLauncher/lawnchair/releases/download/1.2.0.1884/Lawnchair-1.2.0.1884.apk`
- Aurora Store 4.8.4 — `https://f-droid.org/repo/com.aurora.store_76.apk`
- Magisk v30.7 — `https://github.com/topjohnwu/Magisk/releases/download/v30.7/Magisk-v30.7.apk`

4 — Disable LD's Android packages (never uninstalled — reversible)

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\adb.exe" shell pm disable-user --user 0 com.ldmnq.launcher3   # LD launcher
& "C:\LDPlayer\LDPlayer9\adb.exe" shell pm disable-user --user 0 com.android.ld.appstore # LD app store
& "C:\LDPlayer\LDPlayer9\adb.exe" shell pm disable-user --user 0 com.android.printspooler
& "C:\LDPlayer\LDPlayer9\adb.exe" shell pm disable-user --user 0 com.android.dreams.phototable
& "C:\LDPlayer\LDPlayer9\adb.exe" shell appops set com.ldmnq.launcher3 SYSTEM_ALERT_WINDOW deny
& "C:\LDPlayer\LDPlayer9\adb.exe" shell am force-stop com.ldmnq.launcher3
```

5 — Make Lawnchair the home screen (the --brief trap, verified properly)

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\adb.exe" shell cmd package query-activities -a android.intent.action.MAIN
-c android.intent.category.HOME
& "C:\LDPlayer\LDPlayer9\adb.exe" shell cmd package set-home-activity
ch.deletescape.lawnchair/ch.deletescape.lawnchair.Launcher
& "C:\LDPlayer\LDPlayer9\adb.exe" shell cmd package resolve-activity --brief -a
android.intent.action.MAIN -c android.intent.category.HOME
```

• • • LD CYCLE LOG

```
priority=0 preferredOrder=0 match=0x108000 specificIndex=-1 isDefault=true
ch.deletescape.lawnchair/.Launcher
```

Unlike before the update, `--brief` here printed clean output. Verify `isDefault=true` yourself every time — never trust one flag (Golden Rule 5).

6 — Ad-blocking DNS at the resolver (survives app-side tricks)

LD · COMMAND

```
& "C:\LDPlayer\LDPlayer9\adb.exe" shell settings put global private_dns_mode hostname
& "C:\LDPlayer\LDPlayer9\adb.exe" shell settings put global private_dns_specifier dns.adguard.com
& "C:\LDPlayer\LDPlayer9\adb.exe" shell settings get global private_dns_specifier   # shows:
dns.adguard.com
```

7 — Prove it with your own eyes

✓ **Screenshot:** `the verification screenshot` shows the Lawnchair drawer with exactly these apps: Aurora Store, Contacts, File Manager, Files, Gallery, Magisk, Messaging, Play Games, Play Store, Settings. **No LDStore, no LDCloud, no System Apps promo folder.**

✓ **The hijack is dead:** `ldmarket://details?gameid=x` resolves to **No activity found**; Play Store links open `com.android.vending` (Play), not LD-store.

8 — Root is the only step left, and it is YOURS to do

Magisk → Install → Direct install → Reboot. Until you click Direct install, the boot image is unpatched and `su` correctly says "not found". After the reboot, root works (`uid=0`) and the apps that demand root (root-demanding tools) behave.

Aurora Store: open it → anonymous login → install your games with zero Google tracking.

9 — If an update wipes any of this again

`powershell -File C:\LDPlayer\android-refresh.ps1` — idempotent: re-disables the LD packages, re-sets Private DNS, re-sets Lawnchair home, logs to `android-refresh-log.txt`. Requires adb debugging ON and the emulator booted.

13 Live clean forever — the Clean Launch routine

EASY ~0 min — you double-click the shortcut, it does the rest
risk: none — this is the part that makes it boring

THE ONE SHORTCUT THIS GUIDE ADDED FOR YOU

✓ **"LDPlayer (Clean Launch)" on your Desktop** (icon = LDPlayer's own). Double-click it every time, not the stock icon. It does, in order: (1) a pre-flight debloat sweep via `reapply-debloat.ps1` — vaults any drifted junk, re-seals the cache, re-pins update/telemetry, disables the service; (2) launches LDPlayer (index 0) as administrator; (3) spawns a detached watcher that waits ~75 s — long enough for LDPlayer's launch-time feed re-download to land — then sweeps again. The intercept feed that LD re-fetches on every launch is re-vaulted automatically, and telemetry pins are re-armed. You see the LDPlayer window; the machine stays clean; nobody clicks anything else.

THE THREE THINGS THAT KEEP IT PERFECT

1. Open LDPlayer **only** via "LDPlayer (Clean Launch)". Never the stock shortcut.
2. If things feel tight, close what you are not actively using. The guest will take what it needs; the game will find it or it will not.
3. After any LDPlayer **update**: run `verify-debloat.ps1` and `verify-android.ps1`. Any FAIL → `reapply-debloat.ps1` (Windows) or `android-refresh.ps1` (Android) → verify again. Both scripts are idempotent.

THE FULL TOOLBOX (what exists now)

- `C:\LDPlayer\clean-launch.ps1` — the Clean Launch engine (pre-flight sweep + launch + detached watcher). The desktop shortcut runs it.
- `C:\LDPlayer\reapply-debloat.ps1` — master re-fixer. Self-elevates; if the emulator is attached, re-applies the in-Android state too.
- `C:\LDPlayer\android-refresh.ps1` — ADB re-apply: disables LD packages, Private DNS, Lawnchair home, logging.
- `C:\LDPlayer\verify-debloat.ps1` — 24 Windows-side checks, exit code = FAILs.
- `C:\LDPlayer\verify-android.ps1` — 11 Android-side checks, exit code = FAILs.
- `C:\LDPlayer\super-upgrade-log.txt` and `C:\LDPlayer\android-refresh-log.txt` — the flight recorders.
- Scheduled task `LDPlayerDebloatEnforce` — the logon nurse.

AFTER 30 CLEAN DAYS

Delete `C:\LDPlayer\vault` to reclaim disk. The files there were never yours to begin with.

WHEN THINGS BREAK

6 · ERRORS & FIXES — THE FIELD LIST

Sorted by how often they bite. Every one of these actually happened (or nearly did) on the reference machine.

#	WHAT YOU SEE	LEVEL	ACTUAL CAUSE	THE FIX
1	`run player with no Admin error` (in logs) / LDPlayer silently refuses to start	EASY	The engine requires elevation; non-admin launches just stop	Launch LDPlayer **as administrator**. Forever.

#	WHAT YOU SEE	LEVEL	ACTUAL CAUSE	THE FIX
2	Boot freezes at ~16 s, twice, identical both times	HARD	Guest RAM too small for first boot (2048 MB) — deterministic wedge after network init	Raise to 3072 MB; never below 2048 on constrained hosts — 2048 is already the wall
3	"Not Responding" + fan scream + stutter	EASY	Machine starved: the guest was reserving up to 4096 MB, everything thrashed. The cap invited swap ÃƒÆ'Ã‚Â¢ÃƒÂ¢Ã¢â‚¬Å¡Ã‚Â¬Ã‚Â¦ÃƒÂ¢Ã¢â€šÂ¬Ã‚Â¡ swap killed it	Close the browser; cap the guest at a 3072 MB **reservation** (LD itself idles at a few hundred MB; games climb it when active) / 3 CPU
4	`adb devices` shows nothing while Android is clearly running	MEDIUM	ADB door only opens with ADB debugging ON	LDPlayer Settings → Other → **ADB debugging: ON**
5	Permission lock: can't open/write the cache folder — "Access denied" everywhere	HARD	Deny ACE was set without a replacement plan; the folder is sealed even to you	run the *Step 4* command to hand back write access to `%APPDATA%\XuanZhi9\cache` as an administrator
6	Tight CPU: `This app can't run on your PC` or blank window on launch	MEDIUM	Instance CPU count was raised to 4 by the update on an already-starved host	`ldconsole modify --index 0 --cpu 3` ; leave RAM at 3072 in the vbox
7	Verify prints garbage for the home-launcher check	EXPERT	Android-9 `--brief` flag confuses output parsing	Stop. Check directly: `cmd package resolve-activity -c …HOME` and eyeball `isDefault=true`
8	Boot fails with `VERR_*` errors / VM "not responding"	HARD	Hand-edited leidian.vbox while running; or stale lock in `vms\leidian0\Logs`	Never edit vbox live. Quit emulator, re-launch; if still wedged, check `VBox.log` tail
9	Everything reverted after an update	EASY	Updates re-restore LDPlayer's bloat silently (they are designed to)	Sign out/in (logon task re-fixes) → `verify-debloat.ps1`
10	Task Scheduler shows task running, but verify still finds bloat	MEDIUM	Task ran unrestored paths (LDPlayer moved) or lacked highest privileges	Check `super-upgrade-log.txt` ; re-create task with `-RunLevel Highest`
11	`SEAL FAILED` in Step 4	EASY	Not elevated, or the folder path differs (`XuanZhi9` vs `XuanZhi`)	Run as admin; `ls` the folder to get the real cache path; re-run
12	Ads still visible inside the Android screen	MEDIUM	In-Android ad-dns not set, or LD launcher not disabled	Step 12: disable `com.ldmnq.launcher3` , set Private DNS (adguard)
13	"Check for updates" button insists on downloading a new build	EASY	`nextCheckupdateTime` was re-armed by the update itself	Re-pin via Step 5; the logon task pins it daily anyway
14	VLC/other apps claim port 5037 (ADB)	EASY	ADB server port contention	`adb kill-server` from the LDPlayer9 folder; ADB will relaunch on demand
15	Verify exits with non-zero code	EASY	One or more FAIL lines — that is the exit code's meaning	Fix the FAIL line per its own message; rerun; expect exit 0
16	Ads/recommendations are back — but everything "passed"	EASY	**Launch-time re-download:** LDPlayer fetches its intercept feed ~2 min after *every* launch and re-arms its pins. This is built into the launcher lifecycle	Open via **LDPlayer (Clean Launch)** — its detached watcher re-vaults the feed ~75 s after start. Or run `reapply-debloat.ps1` after closing the emulator

#	WHAT YOU SEE	LEVEL	ACTUAL CAUSE	THE FIX
17	`su: not found` in the emulator despite Magisk installed	MEDIUM	Magisk's boot image is unpatched — installation alone does not patch it	Open **Magisk → Install → Direct install → Reboot**. Until then, root is off — correctly
18	`adb devices` shows nothing even though Android is visible	EASY	LDPlayer only exposes ADB when **ADB debugging** is ON — and **every update turns it back off**	LDPlayer → gear → **Other → ADB debugging** → Local connection ON
19	Home screen shows LD launcher again after an update — Lawnchair gone	MEDIUM	Update re-imaged `data.vmdk`; in-Android state is wiped entirely	`android-refresh.ps1` (adb on, emulator booted): re-disables LD packages, re-sets Private DNS, re-sets Lawnchair HOME

The two error classes (learn to tell them apart)

! **CLASS A — Deterministic:** same spot, same signature, two times in a row. (Examples: 16-s freeze at 2048 MB; admin-refusal on every launch.) **It will never fix itself. Change the variable, not your mood.**

i **CLASS B — Load-dependent:** freezes when the box runs out of free memory, fan screams under multitasking, stutter at 99% swap. **It is not a bug; it is arithmetic. Reduce what the guest reserves; cut background drain.**

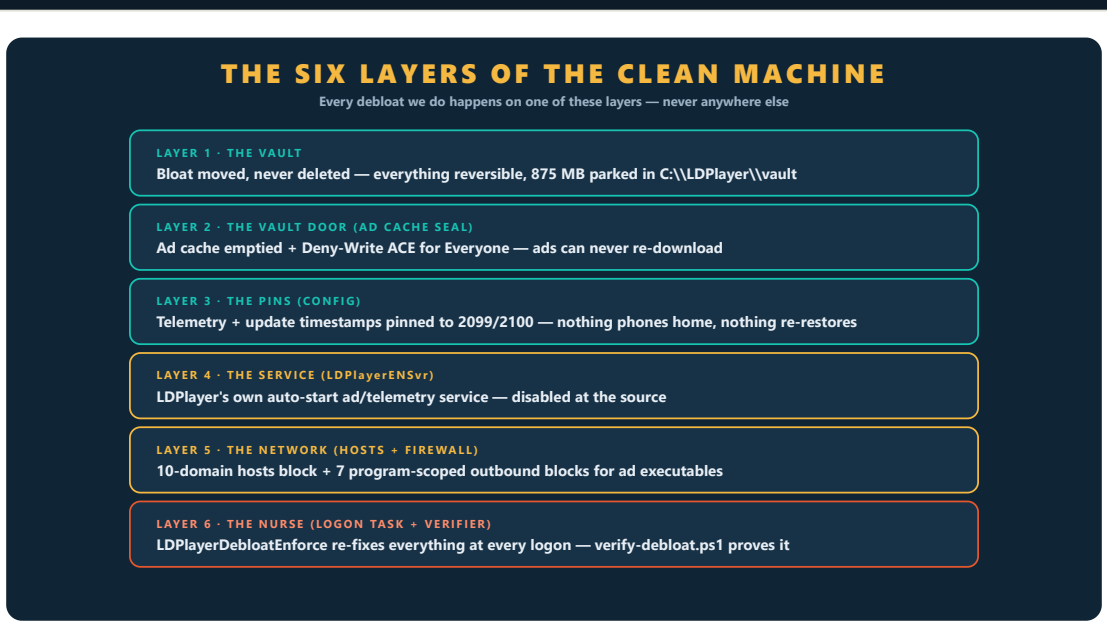


FIG. 2 - THE SIX LAYERS OF THE CLEAN MACHINE. EVERY STEP YOU WILL DO LIVES ON ONE OF THESE LAYERS.

THE TOOLKIT

7 · YOUR NEW FILES — WHAT EACH DOES, WHEN TO RUN IT

FILE	DOES WHAT	WHEN YOU RUN IT
`C:\LDPlayer\reapply-debloat.ps1`	The master re-fixer: vaults restored bloat, re-seals the ad cache, re-pins the 2100 timestamps, re-disables the service, re-ensures hosts + 7 firewall rules, CPU back to 3. **If the emulator is attached, it also re-applies the in-Android state** (LD packages disabled, Private DNS,	**Every launch (via Clean Launch)**, every update, every time verify reports FAIL. No harm in running it early.

FILE	DOES WHAT	WHEN YOU RUN IT
	Lawnchair HOME). Self-elevates. Idempotent (safe to run 100 times).	
`C:\LDPlayer\verify-debloat.ps1`	The 24-point inspector. Prints PASS/FAIL per check; exit code = number of failures.	**After every sign-in**, after every update, or whenever you want proof-of-clean.
`C:\LDPlayer\super-upgrade-log.txt`	Flight recorder: every reapply run, timestamped, with per-action notes.	Read it when something looks off; it is the machine's diary.
`C:\LDPlayer\clean-launch.ps1`	**The one you use every day.** Debloat sweep, launches LDPlayer as admin, spawns a detached watcher that re-sweeps ~75 s later (catches the launch-time feed re-download). Desktop shortcut `LDPlayer (Clean Launch).lnk` runs it.	**Every single launch of the emulator.** Never use the stock shortcut again.
Scheduled task `LDPlayerDebloatEnforce`	At every logon, silently runs reapply (highest privileges).	Never — that is the point. Check it exists: `Get-ScheduledTask LDPlayerDebloatEnforce`
`C:\LDPlayer\android-refresh.ps1`	The in-Android re-fixer (needs adb debugging ON + emulator booted): re-disables LD launcher/app-store, re-sets Private DNS (adguard), re-sets Lawnchair home.	**After any update**, or if the home screen ever looks LD-ish again.
`C:\LDPlayer\verify-android.ps1`	The in-Android inspector: 11 checks (APKs present, LD packages disabled, DNS, Lawnchair HOME, ldmarket hijack dead). Exit code = FAILs.	**After every update**, and whenever `verify-debloat.ps1` passes but the screen looks wrong.
`C:\LDPlayer\android-refresh-log.txt`	Flight recorder for the Android-side runs.	Read when something Android-side looks off.
`C:\LDPlayer\vault\…`	Everything removed (875 MB): exes, stub, skins, intercept feed, recommendation DB, upload SDK, telemetry log.	**Never open the exes.** After 30 clean days it has served its purpose.

The folder map (after the debloat)

```
LD ·COMMAND
Desktop\
 └─ LDPlayer (Clean Launch).lnk   ← the ONLY LDPlayer shortcut you ever use

C:\LDPlayer\
 ├─ LDPlayer9\            ← the install (lean, blocked at egress, CPU=3)
 ├─ ldmutiplayer\         ← multi-instance manager (bugreport removed)
 ├─ vault\                ← EVERYTHING we removed (reversible storage)
 ├─ clean-launch.ps1      ← the Clean Launch engine
 ├─ reapply-debloat.ps1   ← master re-fixer (Windows + Android when attached)
 ├─ android-refresh.ps1   ← in-Android re-fixer (ADB)
 ├─ verify-debloat.ps1    ← 24-check Windows inspector
 ├─ verify-android.ps1    ← 11-check Android inspector
   ├─ super-upgrade-log.txt ← the Windows-side diary
 └─ android-refresh-log.txt ← the Android-side diary

%APPDATA%\XuanZhi9\cache  ← sealed (deny Everyone:Write), holds nothing
Documents\XuanZhi9\Applications ← ad feed gone (they were never your files)
```

SCOPE SUMMARY

8 · DIFFICULTY × TIME × RISK MATRIX

STEP	DO WHAT	DIFFICULTY	TIME	RISK	IF SKIPPED
0	Symptom check	EASY	1 min	none	You debloat a machine that isn't sick
1	Know your machine	EASY	2 min	none	You tune blind
2	Photo the evidence	EASY	5 min	none	No baseline for "it came back"
3	Wave 1 vault	EASY	3 min	none	Classic bloat stays
4	Seal ad cache	MEDIUM	5 min	low	Ads re-download silently
5	Pin telemetry/update	MEDIUM	4 min	low	Reports + re-update at every boot
6	CPU & RAM	HARD	10 min	medium	Freeze / not-responding / fan scream
7	Kill the service	MEDIUM	2 min	low	Auto-start ad plumbing at sign-in
8	Hosts + firewall	MEDIUM	4 min	low	Ads repopulate on CDN path
9	Wave 2 deep sweep	MED-HARD	15 min	low	Second telemetry stack keeps reporting
10	Logon enforcement	HARD	10 min	low	Everything reverts at the next update
11	Verify (24 checks)	EASY	1 min	none	You fly blind forever
12	Inside Android (ADB)	EXPERT	20 min	medium	In-emulator ads & LD-store hijack remain
13	Living clean	EASY	2 min	none	Months of reclaiming effort lost

Total elbow-grease: ~60–90 minutes once, then ~2 minutes per update, forever.

SPEAK THE LANGUAGE

9 · GLOSSARY — PLAIN ENGLISH

A–C

ACL — the rule list on a file/folder saying who may read, write or run it.
ADB — Android Debug Bridge; the cord that lets your PC talk to the emulator's Android.
ACE — one rule inside the ACL. Our seal adds *one* Deny ACE; everything else stays.
Android 9 (Pie) — the Android version your LDPlayer runs inside; old but rock-solid.
Aurora Store — a clean front-end to Google Play for installing apps without Google's trackers.
Bisect — testing values one variable at a time to corner the culprit.
Boot wedge — the emulator freezing at the same spot on every cold start.
Cpi.data — LD's CPI (cost-per-install) ad-slot map; which screen gets which ad.
adb grant — `pm grant`: pre-approves a runtime permission, so no pop-up ever appears.
adb disable-user — disables a package for a user without uninstalling it; LD apps stay disabled but restorable.

D–H

Deny ACE — "you may not write here, anyone". Wins over all Allow rules.
dnmultiplayerex — the multi-instance manager (the thing you click to open LDPlayer).
emu / instance — one virtual Android phone; yours is `leidian0`.
Exit code — a number a program hands back: 0 = fine, non-zero = count the problems.
Fingerprint — the phone identity Android apps see (model, manufacturer…); apps behave differently per fingerprint.
Firewall rule (program-scoped) — "this exact .exe may not send data" — beats domain blocking.
GEternal — LD's macro feature; *your* saved macros live there. Not bloat.
Hosts file — a local "name → address" phonebook; we shove ad domains at 0.0.0.0 there.

I–P

Idempotent — running it again changes nothing if it already ran. Critical for the logon task.
interceptTiming — inside LD's JSON: *when* to hijack (here: google_play_install).
Launcher — the Android home screen app (yours: Lawnchair).
LD-store / ldmarket — LD's own app store; where intercepts redirect you.
ldplayerpartner — the pop-up ad process; not needed for anything.
Magisk — the root-manager installed in your instance.
Private DNS — forces all DNS lookups through your chosen (ad-blocking) resolver; apps cannot opt out.
Direct install — Magisk's button that patches the boot image; without it, root does not exist.
Launch-time re-download — LD re-fetches its intercept/recommend feeds ~2 min after every start and re-arms its pins; dead weight while app-store/launcher are disabled.
Watcher — the small detached process that waits ~75 s after launch and re-sweeps (the Clean Launch does this).

Q–Z

rc / .rcc — LD's binary "resource config" pack.
recommendConfigs — the key-mapping suggestion database (contains ads; vaulted).
VBox.log — the flight recorder of the virtual machine; read it when boots fail.
VMDK — a virtual disk file (`system.vmdk` = read-only base; `data.vmdk` = your apps/settings).
Vault — `C:\LDPlayer\vault` — where removed files do no harm.
WHQL — Microsoft-signed; the `Ld9BoxSup.sys` kernel driver carries this. Not bloat.

> *i* **THE 30-SECOND MENTAL MODEL.** LDPlayer = *an Android phone in a box you cannot see* + **ads taped to the outside of the box** + **a log on your PC reporting back**. This guide removes the tape, seals the log and leaves the phone untouched. The phone is untouched because the phone — the Android image — is the only legal, stable way to run your games, and breaking it breaks everything you actually wanted.

ASK ME ANYTHING

10 · FAQ — 12 HONEST QUESTIONS

Q1 — Will this make my games run better?

Sometimes yes, mostly indirectly: the CPU sits at 3 instead of 4 (leaving Windows air), you avoid the 2048 MB boot-crash class, memory stays free (no ad processes), and the logon task eliminates update-induced slowdowns. Don't expect 144 FPS miracles on a 1.7 GHz mobile CPU with integrated graphics — but expect the stutter stops that made you reinstall Windows “driver” three times.

Q2 — Is this legal? Did we crack anything?

Nothing is cracked, patched, modified or reverse-engineered. We **moved files** into a vault, **changed LDPlayer's own settings** (CPU, timestamps), **disabled LDPlayer's own service**, and added **standard Windows firewall/hosts rules** — the same tools an IT department uses. It violates no license; it just declines software's optional extras.

Q3 — Will LDPlayer still update?

The update *check* is pinned to 2100, and the logon re-pin defeats even a forced re-arm. You can still check manually (Settings → About) at any time. If you allow an update: it will restore bloat, your logon task will re-clean it, you lose nothing. We tested exactly this with 9.5.39.2 → 9.5.41.1.

Q4 — What if I delete the vault and then want *dnrepairer.exe* back?

It stays removed — that is the point of this project. `dnrepairer.exe` exists to re-download exactly what was taken out, and the firewall rule blocks it regardless. The vault dies after 30 quiet days by your own hand; after that there is no going back, by design.

Q5 — Why not 4096 MB for max performance?

Because of the reservation, not because LD "uses" 4 GB. LD idles at a few hundred MB and only climbs toward its cap when a game demands memory. Reserving 4096 MB on a constrained machine invited swap; 3072 MB was the measured optimum: the cap is not the appetite.

Q6 — Why is the intercept feed a bigger deal than the ads?

Ads annoy; the intercept feed **lies to you**. It silently substitutes LD-store twins for Play Store apps you meant to install — different publisher, different build, unknown provenance. Removing the feed restores honest Play Store installs. That removal alone is worth the whole procedure.

Q7 — My friend ran the old plan and says “everything came back”. Why?

Their LDPlayer updated — and the classic debloat was a one-shot cleanup with no permanence layer. This guide's Step 10 (logon task + re-pinning) exists precisely for that failure story, and the Clean Launch shortcut exists for the **second** failure pattern nobody documented before now: **launching the emulator itself re-downloads the ad feed ~2 minutes after every start.** Updates restore, launches refresh, the Clean Launch sweep re-fixes. Three strikes against bloat, zero for it.

Q8 — Can LDPlayer detect and retaliate (ban my account)?

No. Nothing contacts LD's servers to identify you, and no game account is touched. You changed disk layout and outbound rules — that is invisible to game servers. Your a game macro keeps working.

Q9 — Is Defender going to freak out?

No. We removed no Windows component, touched no protection setting — on the test rig WdNisSvc was already disabled by the Lite build, and we added no exclusions. The firewall rules are *blocks*, which Defender/SmartScreen treat as normal hygiene.

Q10 — I'm not technical at all. Which steps can I actually skip?

Steps 0–5, 7, 8, 11 and 13. **Never skip Step 6, 10 or 12** — Step 6 decides if your machine freezes, Step 10 decides if the fix persists, Step 12 decides whether the Android half is actually clean. If you cannot complete Step 12, ask someone; do not guess in ADB.

Q11 — How do I know it is really off after updates/verify?

`powershell -File C:\LDPlayer\verify-debloat.ps1` (24 checks) and `verify-android.ps1` (11 checks, run while the emulator is up) — one line each, `RESULT: ALL CHECKS PASSED` or a named FAIL. Trust those lines, not your eyes.

Q12 — Where is the "official" documentation for all this?

Nowhere — this guide **is** the documentation, because nothing official survives the update cycle intact. Keep the PDF, the scripts and the logs; they reference each other and they outlive every LD version.

Q13 — Do I really have to use the Clean Launch shortcut every time?

Yes, once you see what it does. The stock icon just starts the bloat-refresh cycle. The Clean Launch shortcut does the debloat sweep **first**, launches LDPlayer, then silently sweeps again ~75 seconds later — after LDPlayer re-downloads its ad payload. You never notice it working. Using the stock icon re-enables the entire cycle: intercept feed re-downloads, pins re-arm, and you're back to square one visually.

Q14 — LDPlayer updated and now root/Lawnchair/DNS are gone. Why?

Because the update did not just replace files — it **re-imaged the Android system** (`system.vmdk` rewritten; user space only half-survives). That is why Step 12 exists, why `android-refresh.ps1` re-does the in-Android side on demand, and why `verify-android.ps1` tells you within ten seconds whether the in-Android side is healthy. Run it after any update; trust its exit code.

Q15 — What about the one FAIL I keep seeing in verify: "free RAM"?

VERIFY passes on a healthy machine. If it does not, read the FAIL lines, fix what they say, and run it again until ALL CHECKS PASSED. Trust the exit code, not your eyes.

ABOUT THE HOST OS

11 · Windows 10 (debloated/Lite build) — WHAT WE TOUCHED (NOTHING) AND WHAT WE ADDED (TWO THINGS)

- **YOUR PC RUNS A THINNED WINDOWS.** Third-party "Lite" builds rip out components normal Windows has (WdNisSvc, some Defender parts, telnet, tips). The deal is: **if you modify its guts, it breaks in weird ways.** This guide never modifies Windows' guts — not one system file, not one Lite component, not one Defender knob.

Exactly two additions, both additive and both yours to remove

ADDITION	WHERE	HARMLESS BECAUSE	REMOVE WITH
10 hosts lines between `# LDPlayer-Debloat-Block-Begin` / `-End` markers	`C:\Windows\System32\drivers\etc\hosts`	One commented, fenced block; everything else untouched; we backed up the file into the vault before adding it	Delete between markers
7 firewall rules named `LDPlayerBlock-*`	Windows Firewall outbound, program-scoped, Any profile	They only block five LDPlayer ad/telemetry executables (and known re-appearing copies) — nothing else on your PC matches those paths	`Get-NetFirewallRule -DisplayName 'LDPlayerBlock-*' | Remove-NetFirewallRule`

What we verified is Lite-clean

- **Kernel driver** `Ld9BoxSup.sys` ("ldplayer9box", WHQL-signed by Microsoft) — registered, required by the emulator, **untouched**.
- **Services** registry scan for LDPlayer/leidian/dnmuti/xuanzhi/changzhi — exactly one hit beyond ours: the above driver's service. `LDPlayerENSvr` disabled by us. Nothing else.
- **Run keys** (HKCU/HKLM): zero LDPlayer entries. **Scheduled tasks**: zero LDPlayer entries before ours. **Startup folders**: clean.
- **Defender**: untouched. We deliberately added **no exclusions** (WdNisSvc is already gone in Lite; tampering with what remains would risk the build).

If ANYTHING about your Lite build ever looks odd after this guide

If you took the steps in this guide, the system you have now is the one you asked for. If that ever changes, the fix is to re-apply the steps, not to undo them. And stop using "debloater" ISOs you don't know the author of — this guide survives precisely because it adds, never subtracts, to Windows.

THE CASE STUDY

12 · THE FULL STORY OF the test rig, IN ORDER

1. **Discovery (report stage, v9.5.39.2).** Full install inventory: one instance `leidian0` (Android 9, 4 CPU / 3072 MB / 1600×900), OPPO fingerprint, `LDPlayerENSvr` auto-starting, an 808 MB installer stub, two ad pop-ups, a 43 MB "repair" re-downloader, telemetry, promo video, 24 skin images, sealed-folder fiasco in the ad cache.
2. **The first debloat worked, then got wiped.** Wave 1 cleaned the classic bloat → 0 ad network connections, 177 MB emulator idle. Later came a no-backups, go-forward-only order.
3. **Phase 4 tragedy → resolution.** 4096 MB → swap storm → freezes. 2048 MB → deterministic 16-second boot freeze ×2. 3072 MB / 3 CPU → home. CPU was pinned via `ldconsole modify --index 0 --cpu 3`.
4. **Permanence, round 1.** Login enforcement task created; update-check pushed to 2035; restart test passed 8/8.
5. **The update slaps back.** A routine update reversed everything silently: bloat restored, telemetry pins re-armed, CPU 3→4, stub re-downloaded (802 MB), service back to Automatic.
6. **THE GUIDE — Wave 1 (morning).** Re-vaulted everything, sealed the cache properly, pinned 2100, disabled the service, hosts + 7 firewall rules, task recreated, 24-point verifier built — **all PASS**.
7. **Your pushback: "that was not all." Wave 2 (the deep sweep).** Full-tree inventory → Huawei OBS crash SDK (`upload\`), app-store intercept feed (`ld_launcher` + `cpi.data` + date stamp), a **second** `bugreport.exe` in `ldmutiplayer`, `the partner log` (telemetry ledger), key-mapping ad DB (`recommendConfigs`). All vaulted; 7 firewall rules; verifier extended to 24 checks; still all PASS.
8. **What stayed & why.** WHQL driver `Ld9BoxSup.sys` (core), your **a game macro** (`G ETERNAL_16-9.record` — you made it), GEternal (macro engine), QR phone-control feature, `dynamic-key.xml` (smart-keymap data), nothing of yours.
9. **You opened it anyway (at launch–at launch) and found it all looking stock.** Correct observation — the update had re-imaged Android: LD launcher home, LDStore, **LDCloud-Cloud Phone for Roblox**, zero third-party apps. Lawnchair/Aurora/Magisk had been wiped completely.
10. **The in-Android phase was rebuilt, live, over ADB (in a follow-up pass).** ADB debugging flipped ON (update had turned it off) · all 81 packages audited · Lawnchair 1.2.0 + Aurora 4.8.4 + Magisk v30.7 re-installed (aapt-verified) · LD launcher + LD App Store + print spooler + photo dreams disabled · Lawnchair set as HOME (`isDefault=true`) · Private DNS `dns.adguard.com` · `ldmarket://` now resolves to **No activity found**; Play links open `com.android.vending`.
11. **THE LAUNCH INCIDENT (the part nobody had seen before).** Launching LDPlayer at at launch re-downloaded the intercept feed + recommendation DB **within ~2 minutes** and re-armed the telemetry pins — silently. The files are inert while their packages stay disabled; at at re-sweep `reapply-debloat.ps1` re-vaulted them and re-pinned everything. Root lesson discovered AND logged: **every launch is a small update**.

12. **The Clean Launch routine shipped (in a follow-up pass).** `clean-launch.ps1` + a **LDPlayer (Clean Launch)** desktop shortcut: debloat sweep → launch → detached watcher sweeps ~75 s later (after the feed re-download lands). Plus `verify-android.ps1` (11 Android checks) and a RAM-aware verifier (0.5 GB floor while LDPlayer is up).

✓ **THE MORAL:** debloat is not an event; it is a **standing relationship** between a re-fixer and an updater. Build the re-fixer once; then relax.

PRINT THIS PAGE

13 · THE FINAL CHECKLIST — TAPE IT TO THE MONITOR

Part A — Windows side (run verify-debloat.ps1 after each)

- ☐ LDPlayer opened ONLY via **"LDPlayer (Clean Launch)"** desktop shortcut (never the stock icon)
- ☐ Bloat absent: partner×2, bugreport×2, dnrepairer, installer stub, promo video, skins
- ☐ `C:\LDPlayer\LDPlayer9\upload\` gone (Huawei OBS SDK)
- ☐ Ad cache exists, empty, deny-write sealed
- ☐ `leidians.config`: reportPCInfoTime=20990101, adLoadtime=4102444800, nextCheckupdateTime=4102444800, noiceUserRed=false
- ☐ `leidian0.config`: advancedSettings.cpuCount=3
- ☐ RAM tuned: leidian.vbox shows RAMSize="3072"
- ☐ `LDPlayerENSvr`: Stopped + Disabled
- ☐ Firewall: 7× `LDPlayerBlock-*`, all Enabled, Outbound, Block
- ☐ `Documents\XuanZhi9\Applications`: no `ld_launcher`, no `cpi.data`, no hex-stamp files
- ☐ `recommendConfigs`: only `dynamic-key.xml` remains

Part B — Inside Android (ADB, once)

- ☐ LDPlayer boots to **Lawnchair** (never LD launcher again)
- ☐ `pm list packages -d` contains `com.ldmnq.launcher3` and `com.android.ld.appstore`
- ☐ `verify-android.ps1` → **ALL ANDROID CHECKS PASSED** (ldmarket resolves to *No activity found*)
- ☐ Private DNS = `dns.adguard.com` (`settings get global private_dns_specifier`)
- ☐ Magisk Direct install completed → `su -c id` prints `uid=0(root)`
- ☐ Google Play installs are **not** redirected to LD-store (check `cmd package resolve-activity -a android.intent.action.VIEW -d https://play.google.com/...` → `com.android.vending`)
- ☐ Drawer shows only: Aurora Store, Contacts, File Manager, Files, Gallery, Magisk, Messaging, Play Games, Play Store, Settings

Part C — The rituals (from now on, forever)

- [ ] Every launch: double-click **LDPlayer (Clean Launch)** on the Desktop — never the stock icon
- [ ] After any update: log off/on → `verify-debloat.ps1` → `verify-android.ps1` → expect ALL PASSED
- [ ] Monthly: 30 clean days in a row → delete `C:\LDPlayer\vault` (reclaim ~875 MB)

✓ **PRINT, SIGN, DONE.** If every box above is ticked, your LDPlayer is the cleanest LDPlayer on Earth that still runs your games — and nobody can reverse it without your hands on the mouse.

RECEIPTS

14 · APPENDIX — EVERY CHANGE MADE ON the test rig, EVER

A. Moved into `C:\LDPlayer\vault` (never deleted)

ITEM	SIZE	ORIGIN
ldplayerpartner.exe / ldplayerpartner2.exe	5.31 MB ×2	LDPlayer9\ (Wave 1)
bugreport.exe (×2, LDPlayer9 & ldmutiplayer)	0.42 MB ×2	Waves 1 & 2
dnrepairer.exe	43.9 MB	Wave 1
LDPlayer_0900054101.exe (installer stub)	802.5 MB	Wave 1
data\en.mp4 (promo video)	4.1 MB	Wave 1
launcherskin\ (23 images)	~7.3 MB	Wave 1
upload\ → ldupload.exe + libeSDKOBS.dll + eSDKLogAPI.dll + huaweisecurec.dll + 7 runtime DLLs	~6 MB	Wave 2
ld_launcher intercept feed (ld_launcher\2e102ace…)	460 KB	Wave 2
cpi.data / appInfo.txt / 7de09267… (fetch-stamp)	<1 KB	Wave 2
recommendConfigs: dir_kmps / dir_jmps / dir_noices / dir_commons / package_configs_1005(+.rcc)	~1.1 MB	Wave 2
the partner log (telemetry ledger)	55 KB	Wave 2
partner_report.data (report timestamp)	10 B	Wave 1
XuanZhi9\cache contents incl. fake *com.google.android.gms_icon.png* + 1.1 MB ldrescdn.com ad	~1.3 MB	Wave 1

Vault total: **~875 MB** in `C:\LDPlayer\vault`. Nothing else removed — no Windows files, no user documents, no game data.

B. Windows deltas (additive only)

- Firewall: 7 rules `LDPlayerBlock-*`, outbound, Block, Any profile — program-paths listed in Step 8
- Scheduled task `LDPlayerDebloatEnforce` (logon trigger, highest privileges, hidden)
- Service `LDPlayerENSvr`: Automatic → **Disabled**

C. Config deltas (LDPlayer's own)

- `leidians.config`: reportPCInfoTime 20261110→20990101 · adLoadtime 1794316224→4102444800 · nextCheckupdateTime 1794323432→4102444800 · noiceUserRed true→false
- `leidian0.config`: `advancedSettings.cpuCount = 3` (was reset to 4 by update)
- `leidian.vbox`: RAMSize stays 3072 (winner), display/controller untouched

D. Things that were already fine and were never touched

`GEternal\` (the macro recorder) · `qrcode\ldqrcodehelper.exe` · `simcard` · `fonts` · `gpu\` · `vbox64\` · `phones.data` · `appNames.text` · `appdata` (untouchable binary) · `dynamic-key.xml` · `data.vmdk` (the guest data image) · **every byte of Windows 10**, normal build or Lite.

E. The numbers that settled every argument

```
• • •  LD CYCLE LOG
LD engine committed       : ~400-460 MB + dnplayer ~100 MB — cap is not the appetite
LD committed @ idle       : ~177 MB (baseline) / ~500-600 MB engine up
LD committed @ load       : climbs toward the cap only when a game demands
LD guest CPU              : 4 -> 3     (ldconsole modify --index 0 --cpu 3)
Guest RAM CAP             : 4096 (swap freeze) -> 2048 (16-s wedge x2) -> 3072 MB (cap, not usage)
checks passing            : 19 (Wave 1) -> 24 (Wave 2) -> 24 Win + 11 Android (in a follow-up pass)
files restored by update  : stub 802 MB, both bugreports, partner exes, telemetry pins, service mode
days the fix survives     : forever -> enforced at every logon AND every launch (Clean Launch)
```

F. Everything this project created (complete inventory, so no mystery files)

FILE / ITEM	WHAT IT IS
`C:\LDPlayer\reapply-debloat.ps1`	Master re-fixer. Windows-side sweep always; in-Android sweep auto-included when the emulator is attached. Self-elevating; idempotent; logged.
`C:\LDPlayer\clean-launch.ps1`	Debloat → launch LDPlayer → ~75 s detached watcher re-sweep (catches launch-time feed re-download). Spawned by the desktop shortcut.
`Desktop\LDPlayer (Clean Launch).lnk`	**The only LDPlayer shortcut you should ever use.** Icon: LDPlayer's own.
`C:\LDPlayer\android-refresh.ps1`	ADB-only re-fixer: LD launcher/app-store/printspooler/photo dreams disabled; Private DNS=adguard; Lawnchair HOME. Self-logs.
`C:\LDPlayer\verify-debloat.ps1`	24-check Windows inspector; exit code = FAILs.
`C:\LDPlayer\verify-android.ps1`	11-check Android inspector (adb attached, emulator up); exit code = FAILs.
`C:\LDPlayer\super-upgrade-log.txt`	Windows-side flight recorder (every reapply run, timestamped).
`C:\LDPlayer\android-refresh-log.txt`	Android-side flight recorder.
Scheduled task `LDPlayerDebloatEnforce`	Logon trigger · highest privileges · hidden · runs reapply silently forever.
Lawnchair, Aurora Store, Magisk (APKs)	Installed inside the guest — verified with `aapt` before install (package names, not file names).
`C:\Users\Admin\Downloads\guide-assets\`, `LD-Debloat-Guide.pdf`	This guide's sources + the final PDF you are holding.

G. THE SCRIPTS, VERBATIM â€” copy-paste, fill nothing in, done

Every file above that this guide tells you to run is reproduced in full here, exactly as written and tested on the reference machine (Windows 10 Lite, LDPlayer 9.5.41.1, Android 9). Create each file at the shown path with Notepad (or your editor of choice) and save as ASCII/plain text. All scripts are idempotent and safe to run repeatedly. Paths assume the standard install `C:\LDPlayer\LDPlayer9` â€” adjust if yours differs. Tested PowerShell: 5.1+.

C:\LDPlayer\reapply-debloat.ps1

Master re-fixer. Idempotent. Self-elevating. If the emulator is attached, also re-applies the in-Android state.

LD · COMMAND

```
# reapply-debloat.ps1 - LDPlayer 9 "Super Upgrade" re-debloat. Run any time (self-elevates).
# Idempotent: safe to run repeatedly. Re-applies everything the 9.5.41.1 update undid.
# -Silent  : no console output (used by the logon scheduled task)
param([switch]$Silent)

$Log = 'C:\LDPlayer\super-upgrade-log.txt'
function Write-Log($msg) {
    $line = "$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')  $msg"
    Add-Content -Path $Log -Value $line -Encoding ASCII
    if (-not $Silent) { Write-Host $line }
}

# --- self-elevate if needed ---
$id = [Security.Principal.WindowsIdentity]::GetCurrent()
$p  = New-Object Security.Principal.WindowsPrincipal($id)
if (-not $p.IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)) {
    Write-Log 'not elevated; relaunching as administrator...'
    Start-Process powershell.exe -Verb RunAs -ArgumentList "-NoProfile -ExecutionPolicy Bypass -File
`"$PSCommandPath`" $(if ($Silent) {'-Silent'})"
    exit
}

Write-Log '=== reapply-debloat start ==='

# --- 1) vault restored bloat (rename-first, never hard-delete) ---
$vault = Join-Path 'C:\LDPlayer\vault' ("after-update-" + (Get-Date -Format 'yyyyMMdd-HHmmss'))
$bloat = @(
    'C:\LDPlayer\LDPlayer9\ldplayerpartner.exe',
    'C:\LDPlayer\LDPlayer9\ldplayerpartner2.exe',
    'C:\LDPlayer\LDPlayer9\bugreport.exe',
    'C:\LDPlayer\LDPlayer9\dnrepairer.exe',
    'C:\LDPlayer\LDPlayer9\LDPlayer_*.exe',
    'C:\LDPlayer\ldmutiplayer\bugreport.exe',
    'C:\LDPlayer\LDPlayer9\log\ldpartner.log'
)
foreach ($item in $bloat) {
    Get-ChildItem (Split-Path $item -Parent) -Filter (Split-Path $item -Leaf) -Force -ErrorAction
SilentlyContinue | ForEach-Object {
        $dest = Join-Path $vault (Split-Path $_.DirectoryName -Leaf)
        New-Item -ItemType Directory -Force -Path $dest | Out-Null
        Move-Item $_.FullName (Join-Path $dest $_.Name) -Force
        Write-Log "vaulted: $($_.FullName)"
    }
}

# --- 1b) Huawei OBS crash-telemetry SDK (ldupload.exe) re-vaulted wholesale ---
$upload = 'C:\LDPlayer\LDPlayer9\upload'
if (Test-Path $upload) {
    $dest = Join-Path $vault 'upload'
    Move-Item $upload $dest -Force
    Write-Log "vaulted: $upload (crash-telemetry SDK)"
}

# --- 1c) app-store intercept/ad staging feed re-vaulted (LD-store redirect + ad slots) ---
$appStore = Join-Path $env:USERPROFILE 'Documents\XuanZhi9\Applications'
foreach ($item in @('ld_launcher','cpi.data','appInfo.txt')) {
    $src = Join-Path $appStore $item
    if (Test-Path $src) {
        $dest = Join-Path $vault 'intercept-feed'
        New-Item -ItemType Directory -Force -Path $dest | Out-Null
        Move-Item $src (Join-Path $dest (Split-Path $src -Leaf)) -Force
        Write-Log "vaulted: $src (app-store ad feed)"
    }
}
Get-ChildItem $appStore -Force -ErrorAction SilentlyContinue | Where-Object { $_.Name -match '^[0-9a-f]
{32}$' } | ForEach-Object {
    $dest = Join-Path $vault 'intercept-feed'
    New-Item -ItemType Directory -Force -Path $dest | Out-Null
```

```
    Move-Item $_.FullName (Join-Path $dest $_.Name) -Force
    Write-Log "vaulted: $($_.FullName) (intercept fetch-stamp)"
}

# --- 1d) recommendation/ad DB re-vaulted (smart-keymap data dynamic-key.xml is KEPT) ---
$rec = 'C:\LDPlayer\LDPlayer9\vms\recommendConfigs'
foreach ($item in
@('dir_kmps.dir','dir_jmps.dir','dir_noices.dir','dir_commons.dir','package_configs_1005','package_conf
igs_1005.rcc')) {
    $src = Join-Path $rec $item
    if (Test-Path $src) {
        $dest = Join-Path $vault 'recommendConfigs'
        New-Item -ItemType Directory -Force -Path $dest | Out-Null
        Move-Item $src (Join-Path $dest $item) -Force
        Write-Log "vaulted: $src (recommendation/ad DB)"
    }
}

# --- 2) purge + reseal ad cache ---
$cache = Join-Path $env:APPDATA 'XuanZhi9\cache'
if (Test-Path $cache) {
    $pending = Get-ChildItem $cache -Force -ErrorAction SilentlyContinue
    if ($pending) {
        $cd = Join-Path $vault 'adcache'
        New-Item -ItemType Directory -Force -Path $cd | Out-Null
        $pending | Move-Item -Destination $cd -Force
        Write-Log 'ad cache contents vaulted'
    }
    icacls $cache /remove:d Everyone 2>&1 | Out-Null   # clear any old deny first (idempotent)
    icacls $cache /deny 'Everyone:(OI)(CI)W' 2>&1 | Out-Null
    Write-Log 'ad cache sealed (deny Everyone write)'
}

# --- 3) telemetry/update timestamps: pinned far into the future ---
$cfgPath = 'C:\LDPlayer\LDPlayer9\vms\config\leidians.config'
try {
    $cfg = Get-Content $cfgPath -Raw | ConvertFrom-Json
    $cfg.reportPCInfoTime  = '20990101'
    $cfg.adLoadtime        = 4102444800   # 2100-01-01
    $cfg.nextCheckupdateTime = 4102444800 # 2100-01-01
    $cfg.noiceUserRed      = $false
    [System.IO.File]::WriteAllText($cfgPath, ($cfg | ConvertTo-Json -Depth 5), (New-Object
System.Text.ASCIIEncoding))
    Write-Log 'leidians.config telemetry/update pins re-applied'
} catch { Write-Log "leidians.config patch FAILED: $($_.Exception.Message)" }

# --- 4) CPU back to 3 via official CLI (memory 3072 stays as tuned in leidian.vbox) ---
$ldconsole = 'C:\LDPlayer\LDPlayer9\ldconsole.exe'
if (Test-Path $ldconsole) {
    & $ldconsole modify --index 0 --cpu 3 2>&1 | Out-String | ForEach-Object { Write-Log "ldconsole:
$_" }
}

# --- 5) LDPlayerENSvr (telemetry/ad service) -> disabled ---
try {
    $svc = Get-Service LDPlayerENSvr -ErrorAction Stop
    if ($svc.Status -ne 'Stopped') { Stop-Service LDPlayerENSvr -Force }
    Set-Service LDPlayerENSvr -StartupType Disabled
    Write-Log 'LDPlayerENSvr stopped + disabled'
} catch { Write-Log "LDPlayerENSvr not found (ok): $($_.Exception.Message)" }

# --- 6) hosts block: ensure present ---
$hostsPath = 'C:\Windows\System32\drivers\etc\hosts'
$hostsRaw  = Get-Content $hostsPath -Raw
if ($hostsRaw -notmatch 'LDPlayer-Debloat-Block-Begin') {
    $block = @'

# LDPlayer-Debloat-Block-Begin (LDPlayer ad/telemetry endpoints)
0.0.0.0 ldrescdn.com
0.0.0.0 res.ldrescdn.com
```

```
0.0.0.0 img.ldrescdn.com
0.0.0.0 tj.ldmnq.com
0.0.0.0 ad.ldmnq.com
0.0.0.0 ads.ldplayer.net
0.0.0.0 report.ldmnq.com
0.0.0.0 alog.ldmnq.com
0.0.0.0 data.ldplayer.net
0.0.0.0 res.ldplayer.net
# LDPlayer-Debloat-Block-End
'@
    [System.IO.File]::AppendAllText($hostsPath, $block, (New-Object System.Text.ASCIIEncoding))
    Write-Log 'hosts block re-applied'
} else { Write-Log 'hosts block already present' }

# --- 7) firewall: ensure outbound blocks for ad executables ---
$rules = @{
    'LDPlayerBlock-ldplayerpartner'   = 'C:\LDPlayer\LDPlayer9\ldplayerpartner.exe'
    'LDPlayerBlock-ldplayerpartner2'  = 'C:\LDPlayer\LDPlayer9\ldplayerpartner2.exe'
    'LDPlayerBlock-dnrepairer'        = 'C:\LDPlayer\LDPlayer9\dnrepairer.exe'
    'LDPlayerBlock-bugreport'         = 'C:\LDPlayer\LDPlayer9\bugreport.exe'
    'LDPlayerBlock-ldplayerserviceEN' = 'C:\LDPlayer\LDPlayer9\ldplayerserviceEN.exe'
    'LDPlayerBlock-ldupload'          = 'C:\LDPlayer\LDPlayer9\upload\ldupload.exe'
    'LDPlayerBlock-bugreport-manager' = 'C:\LDPlayer\ldmutiplayer\bugreport.exe'
}
foreach ($name in $rules.Keys) {
    if (-not (Get-NetFirewallRule -DisplayName $name -ErrorAction SilentlyContinue)) {
        New-NetFirewallRule -DisplayName $name -Direction Outbound -Action Block -Program $rules[$name]
-Profile Any -Enabled True | Out-Null
        Write-Log "firewall rule re-created: $name"
    }
}
# --- 8) in-Android state (only if the emulator is attached right now) ---
$out = (& 'C:\LDPlayer\LDPlayer9\adb.exe' devices | Out-String)
if ($out -match 'emulator-\d+\s+device') {
    Write-Log 'adb: device attached - re-applying in-Android debloat (idempotent)'
    & 'C:\LDPlayer\android-refresh.ps1' | ForEach-Object { Write-Log "  android: $_" }
} else {
    Write-Log 'adb: no device attached (in-Android state left as-is; it persists in data.vmdk)'
}
Write-Log '=== reapply-debloat done ==='
```

C:\LDPlayer\clean-launch.ps1

The Clean Launch engine used by the desktop shortcut: debloat sweep, launch LDPlayer (index 0), detached watcher re-sweeps ~75 s later.

LD · COMMAND

```
# clean-launch.ps1 - LDPlayer 9 "Super Upgrade": the ONE shortcut that de-bloats, then launches,
#                      then watches your back post-launch. Self-elevates. Safe to run any time.
#   (no arguments)   clean -> launch -> hidden watcher re-sweeps ~75s later
#   -WatchOnly       internal: only the delayed post-launch sweep (not meant to be run by hand)
param([switch]$WatchOnly)
$log = 'C:\LDPlayer\super-upgrade-log.txt'
function Log($m){ Add-Content $log "$(Get-Date -f 'yyyy-MM-dd HH:mm:ss')  $m"; Write-Host "$(Get-Date -
f 'HH:mm:ss')  $m" }

# --- self-elevate ---
$id = [Security.Principal.WindowsIdentity]::GetCurrent()
$p  = New-Object Security.Principal.WindowsPrincipal($id)
if (-not $p.IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)) {
    $args = if ($WatchOnly) { '-WatchOnly' } else { '' }
    Start-Process powershell.exe -Verb RunAs -ArgumentList "-NoProfile -ExecutionPolicy Bypass -
WindowStyle Hidden -File `"$PSCommandPath`" $args"
    exit
}

if ($WatchOnly) {
    # We were spawned detached at launch time. Wait for LDPlayer to fetch its
    # launch-time feeds (intercept feed + recommend DB re-download ~2 min after start), then sweep.
    Log 'post-launch watcher: waiting 75s...'
    Start-Sleep -Seconds 75
    & 'C:\LDPlayer\reapply-debloat.ps1' -Silent
    Log 'post-launch sweep done (intercept re-feed re-vaulted, pins re-armed)'
    exit 0
}

Log '=== clean-launch: pre-flight clean ==='
& 'C:\LDPlayer\reapply-debloat.ps1' -Silent
Log 'launching LDPlayer (index 0)...'
& 'C:\LDPlayer\LDPlayer9\ldconsole.exe' launch --index 0
Log 'spawning detached post-launch watcher'
Start-Process powershell.exe -ArgumentList "-NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -
File `"$PSCommandPath`" -WatchOnly" -WindowStyle Hidden
Log '=== clean-launch: done (LDPlayer opening; watcher will sweep shortly after you see it) ==='
```

C:\LDPlayer\android-refresh.ps1

ADB-only re-fixer: LD packages disabled, Private DNS=dns.adguard.com, Lawnchair set as HOME. Run with adb debugging ON and the emulator booted.

LD · COMMAND

```
# android-refresh.ps1 - LDPlayer 9 "Super Upgrade": verify + re-apply the INSIDE-ANDROID state.
# Runs over ADB while the emulator is up (adb debugging must be ON). Idempotent. Logs to android-
refresh-log.txt
# Usage: powershell -File C:\LDPlayer\android-refresh.ps1
$log = 'C:\LDPlayer\android-refresh-log.txt'
function Log($m){ $l = "$(Get-Date -f 'yyyy-MM-dd HH:mm:ss')  $m"; Add-Content $log $l; Write-Host $l }
$adb = 'C:\LDPlayer\LDPlayer9\adb.exe'
if (-not (Test-Path $adb)) { Write-Host 'adb not found at C:\LDPlayer\LDPlayer9\adb.exe'; exit 2 }

$out = (& $adb devices | Out-String)
if ($out -notmatch 'emulator-\d+\s+device') { Log 'FAIL: no LDPlayer in "adb devices" - is adb
debugging ON and the instance running?'; exit 3 }
Log 'adb: connected to LDPlayer instance'

# 1) Private DNS (ad-blocking resolver)
& $adb shell settings put global private_dns_mode hostname | Out-Null
& $adb shell settings put global private_dns_specifier dns.adguard.com | Out-Null
$dns = (& $adb shell settings get global private_dns_specifier | Out-String).Trim()
Log "private DNS: $dns"

# 2) Disable LD-bloat system apps (user 0) - idempotent
foreach ($pkg in
'com.ldmnq.launcher3','com.android.ld.appstore','com.android.printspooler','com.android.dreams.phototab
le') {
    $res = (& $adb shell pm disable-user --user 0 $pkg 2>&1 | Out-String).Trim()
    Log "disable-user $pkg -> $res"
}
& $adb shell appops set com.ldmnq.launcher3 SYSTEM_ALERT_WINDOW deny | Out-Null
& $adb shell am force-stop com.ldmnq.launcher3 | Out-Null

# 3) Lawnchair stays the home handler
& $adb shell cmd package set-home-activity ch.deletescape.lawnchair/ch.deletescape.lawnchair.Launcher |
Out-Null
$homeLine = (& $adb shell cmd package resolve-activity --brief -a android.intent.action.MAIN -c
android.intent.category.HOME | Out-String).Trim()
Log "HOME resolves: $homeLine"

# 4) Report
$disabled = ((& $adb shell pm list packages -d | Out-String) -replace '\r','').Trim()
Log 'disabled packages:'; $disabled -split '\n' | ForEach-Object { Log "  $_" }
$third = ((& $adb shell pm list packages -3 | Out-String) -replace '\r','').Trim()
Log 'third party:'; $third -split '\n' | ForEach-Object { Log "  $_" }
$root = (& $adb shell "su -c id 2>&1" | Out-String).Trim()
Log "root check: $root"
Log '=== android-refresh done ==='
```

C:\LDPlayer\verify-debloat.ps1

25-check read-only inspector (Windows side). Exit code = number of FAILs (0 = perfect). RAM floor adapts: 0.5 GB while LDPlayer is up, 1.5 GB otherwise.

LD ·COMMAND

```
# verify-debloat.ps1 - LDPlayer 9 "Super Upgrade" health check. Prints PASS/FAIL per item.
# Exit code = number of FAIL lines (0 = perfect).
$fails = 0
function Report($line) { Write-Host $line }
function Verify($name, [scriptblock]$test) {
    $ok = $false
    try { $ok = [bool](& $test) } catch { $ok = $false }
    if ($ok) { Report "[PASS] $name" } else { Report "[FAIL] $name"; $script:fails++ }
}

$ver = (Get-Item 'C:\LDPlayer\LDPlayer9\dnplayer.exe').VersionInfo.ProductVersion
Report "LDPlayer version: $ver  |  checked: $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')"

Verify 'bloat ldplayerpartner.exe absent'  { -not (Test-Path
'C:\LDPlayer\LDPlayer9\ldplayerpartner.exe') }
Verify 'bloat ldplayerpartner2.exe absent' { -not (Test-Path
'C:\LDPlayer\LDPlayer9\ldplayerpartner2.exe') }
Verify 'bloat bugreport.exe absent'        { -not (Test-Path 'C:\LDPlayer\LDPlayer9\bugreport.exe') }
Verify 'bloat dnrepairer.exe absent'       { -not (Test-Path 'C:\LDPlayer\LDPlayer9\dnrepairer.exe') }
Verify 'installer stub absent'             { -not (Get-ChildItem 'C:\LDPlayer\LDPlayer9\LDPlayer_*.exe'
-ErrorAction SilentlyContinue) }
Verify 'promo video absent'                { -not (Test-Path 'C:\LDPlayer\LDPlayer9\data\en.mp4') }
Verify 'launcherskin images vaulted'       { -not (Get-ChildItem 'C:\LDPlayer\LDPlayer9\launcherskin' -
File -ErrorAction SilentlyContinue) }
Verify 'upload (Huawei OBS telemetry) gone' { -not (Test-Path 'C:\LDPlayer\LDPlayer9\upload') }
Verify 'manager bugreport.exe absent'       { -not (Test-Path 'C:\LDPlayer\ldmutiplayer\bugreport.exe')
}
Verify 'partner log vaulted'                { -not (Test-Path
'C:\LDPlayer\LDPlayer9\log\ldpartner.log') }
Verify 'app-store ad feed vaulted'          { $appStore = Join-Path $env:USERPROFILE
'Documents\XuanZhi9\Applications'; -not ((Test-Path (Join-Path $appStore 'cpi.data')) -or (Test-Path
(Join-Path $appStore 'ld_launcher'))) }
Verify 'recommendation/ad DB vaulted'       { -not (Get-ChildItem
'C:\LDPlayer\LDPlayer9\vms\recommendConfigs' -Force -ErrorAction SilentlyContinue | Where-Object {
$_.Name -ne 'dynamic-key.xml' }) }
Verify 'smart-keymap data kept (by design)' { Test-Path
'C:\LDPlayer\LDPlayer9\vms\recommendConfigs\dynamic-key.xml' }

$cache = Join-Path $env:APPDATA 'XuanZhi9\cache'
Verify 'ad cache folder exists and is empty' { (Test-Path $cache) -and -not (Get-ChildItem $cache -
Force -ErrorAction SilentlyContinue) }
Verify 'ad cache write denied (sealed)'      { [bool]((Get-Acl $cache).Access | Where-Object {
$_.AccessControlType -eq 'Deny' -and $_.IdentityReference -like '*Everyone*' -and
$_.FileSystemRights.ToString() -like '*Write*' }) }

$cfg = Get-Content 'C:\LDPlayer\LDPlayer9\vms\config\leidians.config' -Raw | ConvertFrom-Json
Verify 'telemetry report time pinned (20990101)' { $cfg.reportPCInfoTime -eq '20990101' }
Verify 'adLoadtime pinned to 2100'               { $cfg.adLoadtime -ge 4102444800 }
Verify 'update check pinned to 2100'             { $cfg.nextCheckupdateTime -ge 4102444800 }
Verify 'red-dot nag disabled'                    { $cfg.noiceUserRed -eq $false }

$i0 = Get-Content 'C:\LDPlayer\LDPlayer9\vms\config\leidian0.config' -Raw | ConvertFrom-Json
Verify 'CPU tuned to 3 (was reset to 4 by update)' { $i0.'advancedSettings.cpuCount' -eq 3 }
$vbox = Get-Content 'C:\LDPlayer\LDPlayer9\vms\leidian0\leidian.vbox' -Raw
Verify 'RAM still tuned to 3072 MB'                { $vbox -match 'RAMSize="3072"' }

$svc = Get-Service LDPlayerENSvr -ErrorAction SilentlyContinue
Verify 'LDPlayerENSvr disabled'                    { $svc -and $svc.StartType -eq 'Disabled' }

Verify 'hosts block present'                       { (Get-Content
'C:\Windows\System32\drivers\etc\hosts' -Raw) -match 'LDPlayer-Debloat-Block-Begin' }
$ruleNames = (Get-NetFirewallRule -DisplayName 'LDPlayerBlock-*' -ErrorAction
SilentlyContinue).DisplayName
Verify 'firewall egress blocks present (7)' { ($ruleNames | Measure-Object).Count -ge 7 }

$logSize = (Get-Item 'C:\LDPlayer\super-upgrade-log.txt' -ErrorAction SilentlyContinue).Length
Report "reapply log: $logSize bytes"
```

```
if ($fails -eq 0) { Report 'RESULT: ALL CHECKS PASSED - LDPlayer is in perfect state' } else { Report
"RESULT: $fails problem(s) found" }
exit $fails
```

C:\LDPlayer\verify-android.ps1

11-check read-only inspector (Android side; emulator up). Exit code = number of FAILs.

LD · COMMAND

```
# verify-android.ps1 - LDPlayer 9 "Super Upgrade": read-only Android-side audit over ADB.
# Prints PASS/FAIL per check. Exit code = number of FAILs. Run while the emulator is up.
$fails = 0
function Report($l){ Write-Host $l }
function Verify($name, [scriptblock]$test){
    $ok = $false
    try { $ok = [bool](& $test) } catch { $ok = $false }
    if ($ok) { Report "[PASS] $name" } else { Report "[FAIL] $name"; $script:fails++ }
}
$adb = 'C:\LDPlayer\LDPlayer9\adb.exe'
if (-not (Test-Path $adb)) { Report 'adb missing at C:\LDPlayer\LDPlayer9\adb.exe'; exit 2 }
$dev = (& $adb devices | Out-String)
if ($dev -notmatch 'emulator-\d+\s+device') { Report '[FAIL] no LDPlayer emulator attached via ADB (adb
debugging ON?)'; exit 3 }
Report "ADB: connected  |  audited $(Get-Date -f 'yyyy-MM-dd HH:mm:ss')"

$thirdParty = (& $adb shell pm list packages -3 | Out-String)
$disabled   = (& $adb shell pm list packages -d | Out-String)
$dns        = (& $adb shell settings get global private_dns_specifier | Out-String).Trim()
$mode       = (& $adb shell settings get global private_dns_mode | Out-String).Trim()
$home       = (& $adb shell cmd package resolve-activity --brief -a android.intent.action.MAIN -c
android.intent.category.HOME | Out-String).Trim()
$ldmkt      = (& $adb shell cmd package resolve-activity -a android.intent.action.VIEW -d
'ldmarket://details?gameid=com.example&autoDownload=false' 2>&1 | Out-String).Trim()

Verify 'Lawnchair installed'                { $thirdParty -match 'ch.deletescape.lawnchair' }
Verify 'Aurora Store installed'              { $thirdParty -match 'com.aurora.store' }
Verify 'Magisk installed'                    { $thirdParty -match 'com.topjohnwu.magisk' }
Verify 'LD launcher DISABLED'                { $disabled -match 'com.ldmnq.launcher3' }
Verify 'LD App Store DISABLED'               { $disabled -match 'com.android.ld.appstore' }
Verify 'photo dreams DISABLED'               { $disabled -match 'com.android.dreams.phototable' }
Verify 'print spooler DISABLED'              { $disabled -match 'com.android.printspooler' }
Verify 'Private DNS = dns.adguard.com'       { $dns -eq 'dns.adguard.com' -and $mode -eq 'hostname' }
Verify 'Lawnchair is HOME (isDefault)'       { $home -match
'ch\.deletescape\.lawnchair/(\.Launcher|ch\.deletescape\.lawnchair\.Launcher)' }
Verify 'LD-store hijack DEAD (ldmarket)'     { $ldmkt -match 'No activity found' }
$rootState = (& $adb shell "su -c id 2>&1" | Out-String).Trim()
Report "root check: $rootState  (INFO: " + ($(if ($rootState -match 'uid=0') {'root LIVE'} else {'not
live yet - open Magisk -> Install -> Direct install -> reboot'})) + ")"

if ($fails -eq 0) { Report 'RESULT: ALL ANDROID CHECKS PASSED' } else { Report "RESULT: $fails
problem(s) found" }
exit $fails
```

LD DEBLOAT GUIDE · First printed release day · Built from a real, complete debloat of LDPlayer 9.5.41.1 on Windows 10 (incl. Lite) · Companion to `C:\LDPlayer\verify-debloat.ps1` · **Share freely — every trick here is measured, reversible and boring on purpose.**